
Laboratory of Data Science

Business intelligence, ETL, data warehousing, OLAP

Francesco Secoli • Academic year 2025/2026

Disclaimer. Derived from the *Laboratory of Data Science* course materials (Academic Year 2025/2026), MSc in Data Science & Business Informatics, University of Pisa.

These notes are open educational content created by a student. They are **not an academic source** and may contain inaccuracies. You may freely share, modify, and reuse this material for educational and non-commercial purposes with appropriate attribution. The content is my personal interpretation of the professor's course materials and should not replace official teaching resources. I assume no responsibility for any errors or misinterpretations.

These notes were produced with an **AI-in-the-middle** workflow: a first human pass, then **Claude Code** to support formulation, understanding, and rewriting, followed by a final human review.

If you find errors, have suggestions, or spot unintentionally included copyrighted material (which I will promptly remove on notification), contact me at sclfnc@proton.me.

Contents

1	Business Intelligence Architectures	4
1.1	Data Sources	4
1.2	Extract, Transform and Load	4
1.3	Data Warehouse	5
1.4	Data Warehouse Servers	6
1.5	Mid-tier Servers	6
1.6	Front-end Applications	7
2	Data Access: Files	8
2.1	Locating Files	8
2.2	Tabular Formats	9
2.3	JSON Format	11
2.4	XML Format	11
3	Relational Database Access	13
3.1	Connection Architecture	13
3.2	ODBC	14
3.3	Python DB-API	15
3.4	Programming with pyodbc	15
3.5	Prepared Statements and Security	17
3.6	Data Type Mapping	18
3.7	Metadata Access	19
3.8	Project Setup: Python Environment	20
4	Microsoft SQL Server	22
4.1	Editions and Licensing	22
4.2	Architecture of the Suite	23
4.3	System Databases	24
4.4	Namespace and Object Identification	24
4.5	Data Import and Export	25
4.6	Linked Servers	26
4.7	Connecting to SQL Server	27
5	ETL – Extract, Transform and Load	28
5.1	Phases of the ETL Process	28
5.2	SQL Server Integration Services	30
5.3	Deployment and Execution	32
5.4	Change Data Capture	33
5.5	Slowly Changing Dimensions	33
5.6	Worked Exercises	34
5.7	Date Dimension: A Worked Case Study	40
5.8	SSIS Quick Reference	42
6	Data Warehousing and OLAP	43
6.1	Dimensional Modeling	43
6.2	Storage Technologies	45
6.3	OLAP Concepts	45
6.4	Cuboids in SQL	46

6.5	OLAP Architectures	48
6.6	Data Warehouse Schema: DDL and Load Order	49
7	SQL Server Analysis Services	52
7.1	SSAS Architecture	52
7.2	Developing Multidimensional Projects	53
7.3	OLAP Storage in SSAS	54
7.4	Advanced Features	55
7.5	MDX: MultiDimensional eXpressions	56
7.6	SSAS Build Checklist	58
7.7	Power BI	59
7.8	Worked Exercises	60
7.9	MDX Pattern Reference	65

1 Business Intelligence Architectures

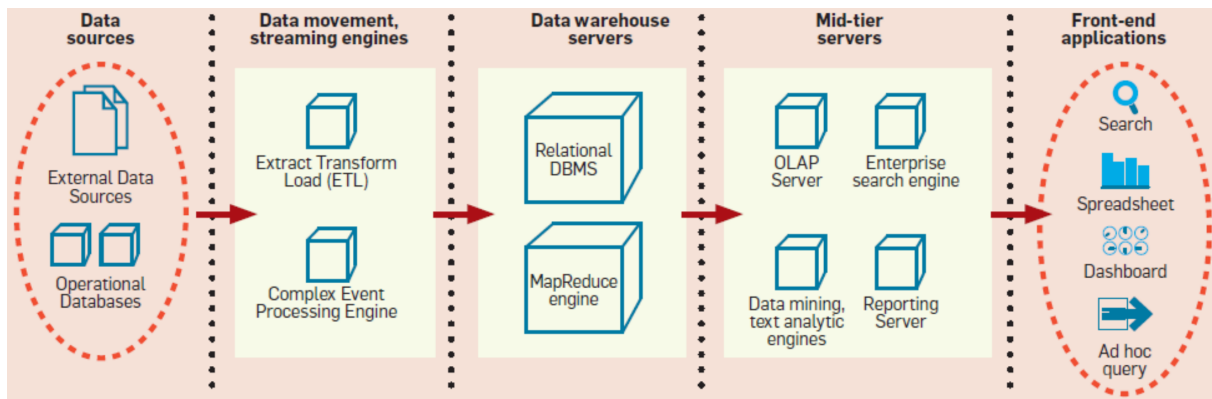


Figure 1.1. Architecture of a BI system.

A Business Intelligence (BI) system turns raw operational data into information that supports decision-making. Its architecture is organized into five interconnected functional layers (Figure 1.1):

1. the **data sources** feed the system;
2. the **ETL** layer extracts, transforms, and loads the data;
3. the **data warehouse** stores it in integrated form;
4. the **mid-tier servers** run advanced analysis;
5. the **front-end applications** present the results.

1.1 Data Sources

Data sources are the entry point of the BI architecture. An organization typically manages several operational sources spread across its departments, complemented by external feeds.

The data comes in heterogeneous formats: relational, multidimensional, documents, time series, spatial data, and real-time streams. This variety raises significant integration problems.

1.2 Extract, Transform and Load

ETL (Extract, Transform, Load) is the process that extracts data from heterogeneous sources, transforms it to guarantee quality, consistency, and conformance to the target schema, and loads it into the data warehouse.

The ETL process sits at the core of how the data warehouse is fed. Its three phases run in sequence: **extraction** retrieves the data from the sources; **transformation** cleans, normalizes, and enriches it; **loading** inserts it into the target structures.

Visual tools such as *SQL Server Integration Services (SSIS)* simplify the design of complex ETL pipelines, letting a developer define transformation flows through drag-and-drop graphical interfaces.

Traditional ETL runs in **batch** mode: the data is processed within predefined time windows (nightly, weekly). This mode introduces latency but guarantees stability and control.

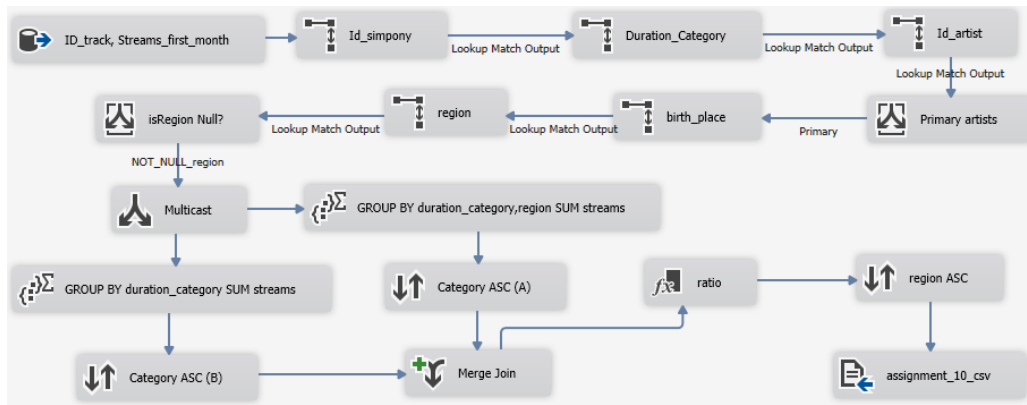


Figure 1.2. An ETL pipeline.

Modern scenarios often call for **incremental** or **real-time** processing. Incremental ETL processes only the data changed since the last run, cutting both time and resources.

Complex Event Processing (CEP) is a paradigm that analyzes continuous event streams in real time, identifies complex patterns through persistent queries, and produces immediate responses toward alerting systems, dashboards, or databases.

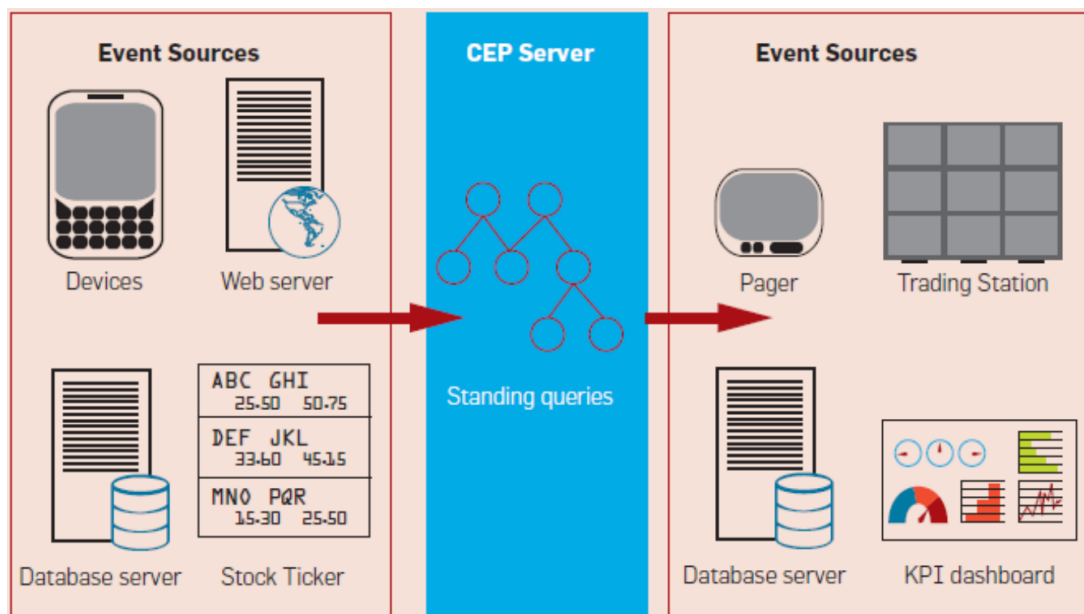


Figure 1.3. CEP architecture: continuous streams from heterogeneous sources (IoT, web, databases) are processed by persistent queries to generate alerts and real-time updates.

1.3 Data Warehouse

A **data warehouse** is a collection of data that supports decision-making processes, defined by four fundamental properties:

- *Subject-oriented*: the data is organized around the key concepts of the business (customers, products, sales) rather than around operational applications.
- *Integrated*: data from heterogeneous sources is reconciled, made uniform in its encodings,

and rendered consistent.

- *Time-variant*: the data warehouse preserves the historical record of the data, enabling trend analysis and comparisons over time.
- *Non-volatile*: loaded data is neither modified nor deleted; the warehouse operates read-only, with periodic insertions.

The term *data warehousing* denotes the entire process of designing, building, feeding, and using a data warehouse. Its typical structure follows a **dimensional schema** optimized for analytical queries.

1.4 Data Warehouse Servers

The servers that host the data warehouse fall into two main categories: RDBMS optimized for analytical workloads, and distributed platforms for big data.

Relational databases built for data warehousing implement optimizations specific to analytical workloads, which are marked by complex queries over large volumes with few writes:

- **Star-join optimization**: execution plans tuned for joins on star schemas, with efficient access to the fact table through the dimensional keys.
- **Bitmap index**: compressed indexes particularly effective on low-cardinality columns (e.g. gender, state, category), which are common in dimensions.
- **Partitioning**: horizontal partitioning of tables by date or another key, which limits scans to the relevant partitions only.
- **Materialized views**: pre-computed views that store the results of frequent aggregations, refreshed periodically.
- **Columnar storage**: organizing data by column instead of by row, which reduces I/O for queries that touch few attributes across many rows.

For volumes that exceed the capacity of individual servers, distributed platforms offer parallel processing on clusters of commodity hardware.

The **MapReduce** paradigm breaks processing into phases: *Map* processes the input chunks in parallel, producing key-value pairs; *Shuffle* redistributes the data, grouping it by key; *Reduce* aggregates the values associated with each key. This model scales horizontally by adding nodes to the cluster.

1.5 Mid-tier Servers

The mid-tier layer hosts services that turn the raw warehouse data into analytical information ready for consumption. It comprises OLAP servers, reporting engines, data mining engines, and enterprise search engines.

OLAP (OnLine Analytical Processing) provides a **multidimensional** view of the data, organized into structures called *cubes* that allow interactive navigation along multiple dimensions of analysis.

The user navigates the cube through intuitive operations: **drill-down** to increase detail, **roll-up** to aggregate, **slice** and **dice** to select subsets, and **pivot** to reorganize the display.

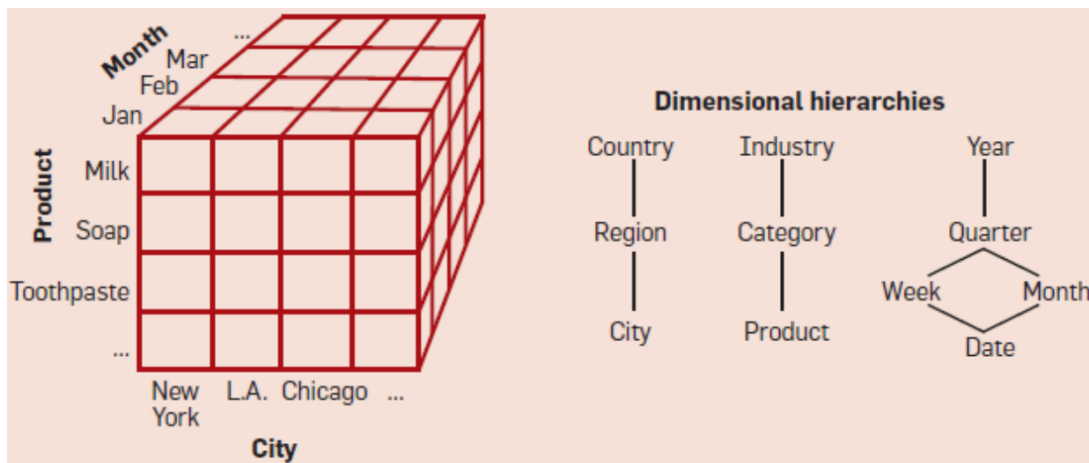


Figure 1.4. Representation of a multidimensional cube: measures (e.g. sales) can be analyzed along hierarchical dimensions (time, product, location).

Reporting servers manage the entire report lifecycle: parametric definition, scheduled execution, multi-format rendering (PDF, Excel, HTML, web), and automatic distribution via email or portals. The data is retrieved from the data warehouse or directly from the OLAP servers.

Data mining servers extract patterns and predictive models from large data volumes. They support algorithms for classification, clustering, association, regression, and anomaly detection. Integration with the data warehouse allows mining techniques to be applied to consolidated historical data.

Enterprise search engines index heterogeneous content (internal web pages, documents, email, structured data), enabling unified keyword search. Full-text indexing complements structured SQL/MDX queries.

1.6 Front-end Applications

Front-end applications form the interface between the BI system and its end users, offering access modes that differ by user profile and skill.

- **Spreadsheets and pivot tables:** interactive navigation of multidimensional data in a familiar environment (Excel, Google Sheets). Pivot tables let the user place dimensions on rows and columns, apply filters, and expand or collapse hierarchies.
- **Dashboards and portals:** concise visualizations with KPIs, charts, and indicators that update automatically. Enterprise portals centralize access to reports and analytics.
- **Ad-hoc query tools:** interfaces for advanced users who build custom queries without resorting to SQL.
- **Vertical applications:** packaged solutions for specific domains, such as CRM analytics, supply chain optimization, financial reporting, and sentiment analysis.

- **Data storytelling:** tools that combine visualizations, annotations, and narrative to communicate insights in a form decision makers can grasp.

2 Data Access: Files

Data access is the first step of any processing pipeline. Before transforming, analysing, or loading information into a data warehouse, the sources must be located and their format interpreted.

2.1 Locating Files

Location uniquely identifies a data resource within a storage system. The mechanisms differ according to whether the resource is local or remote.

2.1.1 Local File Systems

A local file system provides a logical abstraction over persistent mass storage. Its hierarchical structure organises resources into a tree of directories and files, with a root that marks the starting point of navigation.

The **path** uniquely identifies a resource inside the file system. Its syntax varies across operating systems:

```
Windows: C:\Users\analyst\data\sales.csv  
Linux/macOS: /home/analyst/data/sales.csv
```

At the physical level, the file system operates on disk partitions organised into blocks. The file-system driver supplies the logical abstraction, mapping high-level operations (open, read, write) onto block I/O operations. This separation lets applications ignore the details of the underlying hardware.

2.1.2 Distributed File Systems

A distributed file system extends the hierarchical view to resources that physically reside on remote machines. The user perceives a single directory tree, even though the data is spread across different servers. Examples include NFS (Network File System) in Unix environments and SMB/CIFS shares under Windows.

Distributed file systems built for big data, such as HDFS (Hadoop Distributed File System), partition large files across many nodes of a cluster and provide fault tolerance through block replication.

2.1.3 Remote Access Protocols

When a distributed file system is unavailable or impractical, remote files are reached through explicit network protocols. In this scenario a local copy is usually made before the data is processed.

Remote resources are identified by a **URL** (Uniform Resource Locator):

URL syntax:

`scheme://user:password@host:port/path`

The components are: *scheme* (protocol), optional credentials, *host* (server name or IP), *port* (listening port), and *path* (path to the resource).

The main file-transfer protocols are:

- **HTTP/HTTPS:** the standard, stateless web protocol, used to reach resources through REST APIs or by direct download. HTTPS adds TLS encryption.
- **FTP/SFTP:** protocols dedicated to file transfer. FTP operates in cleartext; SFTP (SSH File Transfer Protocol) encrypts the entire session.
- **SCP** (Secure Copy): direct copy between remote hosts over an SSH channel:


```
scp data.csv user@server.example.com:/data/imports/
```
- **Cloud Storage API:** services such as Amazon S3, Azure Blob Storage, and Google Cloud Storage expose REST APIs for programmatic access to distributed object storage.

2.2 Tabular Formats

Tabular data organises information into a two-dimensional structure of rows (records, instances) and columns (attributes, fields).

2.2.1 CSV: Comma Separated Values

The **CSV** format stores each record on one line of text, with attribute values separated by a delimiter character (commonly a comma, semicolon, or tab).

```
name,surname,age,occupation
Mario,Bianchi,23,Student
Luigi,Rossi,30,Engineer
Anna,Verdi,50,Teacher
```

The format is simple and readable, but it carries ambiguities that require extra conventions:

- **Quoting:** when a value contains the delimiter or special characters, it is wrapped in double quotes. For example, "Rossi, Mario" for a compound surname.
- **Escaping:** a double quote inside a quoted field is written by doubling it: "Detto ""il capo""".
- **Missing values:** represented by an empty field, the string `null`, `NA`, `?`, or another documented convention.
- **Header:** the first line may hold the column names; without a header, external metadata is required.

Common variants. TSV (Tab Separated Values) uses the tab as delimiter, avoiding conflicts with commas in the data. The semicolon delimiter is widespread in European locales, where the comma is the decimal separator.

2.2.2 FLV: Fixed Length Values

The **FLV** (Fixed Length Values) format assigns a fixed number of characters to each column, regardless of the actual length of the value.

Mario	Bianchi	23Student
Luigi	Rossi	30Engineer
Anna	Verdi	50Teacher

This structure trades off against delimited formats:

- *Advantage*: direct access to any record without sequential scanning; the position of record n is computable as $n \times \text{row_length}$.
- *Disadvantage*: wasted space for short values (padded with spaces) and truncation of long ones.

FLV is still common in legacy systems, mainframes, and banking interfaces.

2.2.3 Metadata for Tabular Data

Metadata describe the structure and properties of the data: table name, column names and types, integrity constraints. Metadata can be stored in several ways:

- **Header in the file**: one or more initial lines hold the column names and optionally their types.
- **Separate file**: a schema file (e.g. `.schema`, `.json`) accompanies the data file.
- **Dedicated section**: self-describing formats integrate metadata and data in the same file.

2.2.4 ARFF: Attribute-Relation File Format

The **ARFF** format, developed for the Weka data mining suite, integrates metadata and data in a single structured file. The header section defines the relation and its attributes; the data section holds the instances.

```
@relation employees

@attribute name string
@attribute surname string
@attribute age integer
@attribute occupation {Student,Engineer,Teacher}

@data
Mario,Bianchi,23,Student
Luigi,Rossi,?,Engineer
Anna,Verdi,50,'PhD student'
```

Supported attribute types:

- **numeric, real, integer**: numeric values
- **string**: arbitrary text strings
- **{val1, val2, ...}**: enumeration (nominal attribute)
- **date "format"**: dates with a Java SimpleDateFormat pattern

Syntactic conventions: comments with `%`, missing values with `?`, and single-quote quoting for strings that contain spaces or special characters.

2.3 JSON Format

JSON (JavaScript Object Notation) is a lightweight, readable, language-independent data-interchange format. Born in the JavaScript context, it is now the de facto standard for web APIs and configuration files.

2.3.1 Structure and Syntax

JSON supports two composite structures and four primitive types:

- **Object:** an unordered collection of key-value pairs, enclosed in braces { }.
- **Array:** an ordered sequence of values, enclosed in square brackets [].
- **Primitives:** strings (in double quotes), numbers, booleans (**true**, **false**), and **null**.

```
{
  "employees": [
    {"name": "Mario", "surname": "Bianchi", "age": 23},
    {"name": "Anna", "surname": "Verdi", "age": 50}
  ],
  "department": "Engineering",
  "active": true
}
```

2.3.2 JSON for Tabular Data

Tabular data is typically represented as an array of objects, where each object corresponds to a record:

```
[
  {"name": "Mario", "age": 23, "occupation": "Student"},
  {"name": "Luigi", "age": 30, "occupation": "Engineer"},
  {"name": "Anna", "age": 50, "occupation": "Teacher"}
]
```

Unlike CSV, JSON is self-describing: the attribute names are explicit in every record. This introduces redundancy but removes ambiguity, and it allows records with different attributes (flexible schema).

JSON Lines (JSONL). For large datasets, the JSON Lines format stores one JSON object per line, which eases streaming processing and incremental appends:

```
{"name": "Mario", "age": 23}
{"name": "Anna", "age": 50}
```

2.4 XML Format

XML (eXtensible Markup Language) is a metalanguage for defining custom markup languages. Unlike HTML, which has a fixed set of tags, XML lets a designer define vocabularies specific to any application domain.

The *markup* consists of tags that annotate the data with their semantic meaning. XML solves the problem of exchanging data between heterogeneous applications: it gives a self-describing, extensible, and validatable format in which every element is marked with its own role.

2.4.1 Structure of an XML Document

An XML document consists of an optional **prolog** and a hierarchy of nested elements.

```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <cd id="1">
    <artist>Iron Maiden</artist>
    <album>Killers</album>
    <year>1981</year>
    <tracks>
      <track number="1">The Ides of March</track>
      <track number="2">Wrathchild</track>
    </tracks>
  </cd>
</catalog>
```

The **prolog** `<?xml ... ?>` declares the XML version (mandatory when the prolog is present) and the character encoding (UTF-8 by default).

Elements are delimited by an opening tag `<name>` and a closing tag `</name>`. Empty elements use the shorthand syntax `<name />`. Every document has exactly one **root element** that contains all the others.

Attributes specify properties of elements in the form `name="value"` inside the opening tag. The order of attributes is not significant.

2.4.2 Text and Special Characters

Characters with a syntactic meaning in XML require escape sequences (entity references):

```
&lt; → <      &gt; → >
&amp; → &     &quot; → "
&apos; → '    &copy; → ©
```

For blocks of text that contain many special characters, a **CDATA** section avoids escaping:

```
<script><![CDATA[
  if (a < b && c > d) { process(); }
]]></script>
```

2.4.3 Tabular Data in XML

There are two main approaches to representing tabular data in XML.

Attribute style: the values are attributes of the row element:

```
<employees>
  <employee name="Mario" surname="Bianchi" age="23" />
  <employee name="Anna" surname="Verdi" age="50" />
</employees>
```

Element style: the values are nested elements:

```
<employees>
  <employee>
    <name>Mario</name>
    <surname>Bianchi</surname>
    <age>23</age>
```

```
</employee>
</employees>
```

The attribute style is more compact; the element style handles complex, multiline, or markup-bearing values better. Schema metadata can be included in the document itself or in a separate XSD file.

2.4.4 XRFF: XML for Machine Learning

The **XRFF** (eXtensible attribute-Relation File Format) is the XML version of ARFF, developed for Weka. Compared to textual ARFF, it offers greater expressiveness: it supports instance weights, an explicit marking of missing values, and additional data types.

```
<dataset name="employees">
  <header>
    <attributes>
      <attribute name="name" type="string"/>
      <attribute name="age" type="numeric"/>
      <attribute name="role" type="nominal">
        <labels>
          <label>Junior</label>
          <label>Senior</label>
        </labels>
      </attribute>
    </attributes>
  </header>
  <body>
    <instances>
      <instance>
        <value>Mario</value>
        <value>23</value>
        <value>Junior</value>
      </instance>
      <instance>
        <value>Anna</value>
        <value missing="yes"/>
        <value>Senior</value>
      </instance>
    </instances>
  </body>
</dataset>
```

3 Relational Database Access

Programmatic access to relational databases (RDBMS) relies on standardized protocols and APIs that let applications connect, query, and manipulate the data. This part introduces the connection standards and then works through their practical use with Python, a language widely adopted in data science and ETL.

3.1 Connection Architecture

3.1.1 Client-Server Model

Interaction with an RDBMS follows the client-server paradigm: the client application opens a network connection to the database server, sends SQL commands, and receives the results.

This separation lets several clients access the same data concurrently, while the server handles transactions, locking, and query optimization.

The workflow has three phases:

1. **Connection:** locate the server (hostname, port), open the communication channel, authenticate with user credentials, and select the database.
2. **Querying:** send SQL commands, whether read queries (SELECT), modifications (INSERT, UPDATE, DELETE), schema definitions (CREATE, ALTER), or stored procedures.
3. **Result processing:** receive the result set, scan it row by row, and read the values and column metadata.

3.1.2 Connection Standards

Uniform access to heterogeneous RDBMS rests on standards that define common interfaces, insulating the application from the peculiarities of any single database:

- **ODBC** (Open DataBase Connectivity): a cross-platform standard defined by Microsoft and adopted universally. It provides a C API for access to any tabular data source. It ships natively on Windows; on Linux it comes through implementations such as unixODBC and iODBC.
- **JDBC** (Java DataBase Connectivity): a Java standard (part of Java SE) that defines interfaces for database connection. Each DBMS ships a JDBC driver that implements those interfaces.
- **OLE DB**: a Microsoft standard built on COM (Component Object Model), designed to reach tabular, hierarchical, and multidimensional data. It is more powerful than ODBC but confined to the Windows ecosystem.
- **ADO.NET**: a .NET framework that abstracts data access through specific providers. It supports a connected mode (DataReader) and a disconnected mode (DataSet).

3.2 ODBC

ODBC (Open DataBase Connectivity) offers a uniform interface that hides the differences between DBMS. An application written against ODBC can connect to SQL Server, Oracle, MySQL, PostgreSQL, or any other source simply by swapping the driver, with no change to the code.

3.2.1 ODBC Architecture

The ODBC architecture has three components:

- **Application:** the program that uses the ODBC API to reach the data.
- **Driver Manager:** a system library that receives the application's calls, dynamically loads the appropriate driver, and routes the requests.
- **ODBC driver:** a DBMS-specific module that translates the generic ODBC calls into the native protocol of the target database.

DSN configuration. Data sources are registered with the system as a **DSN** (Data Source Name), which binds a logical name to the connection parameters. On Windows this uses the ODBC Data Source Administrator; on Linux it uses the files `/etc/odbc.ini` (system DSN) and `~/.odbc.ini` (user DSN), with the drivers declared in `odbcinst.ini`.

3.3 Python DB-API

Python defines a standard specification for its database-access modules, called **DB-API 2.0** (PEP 249). This specification guarantees uniformity: whatever the DBMS or the chosen module, the code keeps the same structure and calls the same methods.

The portability reaches many databases: SQL Server, Oracle, MySQL, PostgreSQL, SQLite, IBM DB2, and NoSQL databases such as MongoDB and Cassandra through compatible modules.

3.3.1 Available Modules

Each DBMS has several modules that conform to the specification. For MySQL, for instance:

- `MySQLdb`: a C binding for the native MySQL library
- `PyMySQL`: a pure-Python implementation
- `mysql-connector-python`: the official Oracle driver
- `pyodbc`: access through an ODBC driver

The choice depends on performance requirements, dependencies, and portability. For SQL Server, `pyodbc` is the standard choice in a data science setting.

3.3.2 Standard Interface

The DB-API specification defines objects and methods common to every conforming module:

Function/Method	Description
<code>connect(params)</code>	Create and return a Connection object
<code>conn.close()</code>	Close the connection and release its resources
<code>conn.commit()</code>	Commit the current transaction
<code>conn.rollback()</code>	Roll back the current transaction
<code>conn.cursor()</code>	Create a Cursor object to run queries
<code>cursor.execute(sql, [params])</code>	Run a SQL query
<code>cursor.executemany(sql, seq)</code>	Run the query for each item of the sequence
<code>cursor.fetchone()</code>	Return the next row of the result set
<code>cursor.fetchmany(n)</code>	Return the next <i>n</i> rows
<code>cursor.fetchall()</code>	Return all remaining rows
<code>cursor.description</code>	Column metadata of the result set
<code>cursor.rowcount</code>	Number of rows affected by the last operation
<code>cursor.close()</code>	Close the cursor

3.4 Programming with pyodbc

The `pyodbc` module implements the DB-API 2.0 specification using ODBC as its transport protocol. It connects Python applications to any data source that has an ODBC driver.

3.4.1 Workflow

The standard data-access pattern runs in five steps:

1. Import the module
2. Open the connection to the database
3. Run SQL queries through a cursor
4. Process the results
5. Close the cursor and the connection

3.4.2 Connecting to the Database

The connection needs a **connection string** that names the driver, server, database, and credentials:

```
import pyodbc

conn = pyodbc.connect(
    'DRIVER={ODBC Driver 17 for SQL Server};'
    'SERVER=tcp:lds.di.unipi.it;'
    'DATABASE=Foodmart;'
    'UID=student;'
    'PWD=password'
)
cursor = conn.cursor()
```

The general form of the connection string is:

DRIVER={name}; SERVER=host; DATABASE=db; UID=user; PWD=pwd

For connections that use integrated Windows authentication, UID and PWD are omitted and `Trusted_Connection=yes` is added instead.

3.4.3 Running Queries

SQL queries run through the cursor's `execute()` method:

```
# Read query
cursor.execute("SELECT name, age FROM students WHERE age > 20")

# Modification query (requires explicit commit)
cursor.execute("UPDATE students SET age = age + 1 WHERE id = 42")
conn.commit()
```

Modification operations (INSERT, UPDATE, DELETE) do not persist until `commit()` is called. On error, `rollback()` discards the pending changes.

3.4.4 Processing the Results

Two approaches process the result set.

Full fetch loads every row into memory as a list:

```
cursor.execute("SELECT name, department FROM employees")
rows = cursor.fetchall()
for row in rows:
```



```
print(row[0], row[1])          # access by positional index
print(row.name, row.department) # access by column name
```

Cursor iteration stays memory-efficient for large result sets:

```
cursor.execute("SELECT name, department FROM employees")
for row in cursor:
    print(row.name, row.department)
```

Iteration is preferable when the result set may not fit in memory, or when processing should begin before every row has arrived.

3.4.5 Modification Operations

INSERT, UPDATE, and DELETE report the number of rows they touched through the **rowcount** attribute:

```
cursor.execute("DELETE FROM products WHERE discontinued = 1")
print(f"Rows deleted: {cursor.rowcount}")
conn.commit()
```

3.4.6 Resource Management

Releasing resources once they are no longer needed is essential. The traditional approach requires explicit calls:

```
cursor.close()
conn.close()
```

The recommended approach uses the **context manager** (the **with** construct), which guarantees closure even when an exception is raised:

```
with pyodbc.connect(connection_string) as conn:
    with conn.cursor() as cursor:
        cursor.execute("SELECT * FROM products")
        for row in cursor:
            print(row.product_name)
# Connection and cursor closed automatically
```

3.5 Prepared Statements and Security

3.5.1 Parameterized Queries

When many queries share an identical structure but differ in their parameters, computing an execution plan for each one wastes work. **Prepared statements** solve this by using placeholders for the parameters.

Inefficient approach: a full query for every insertion:

```
INSERT INTO employees (id, name) VALUES (1, 'Luigi Rossi')
INSERT INTO employees (id, name) VALUES (2, 'Mario Bianchi')
-- The DBMS compiles N separate execution plans
```

With prepared statements, the plan is compiled once and reused:

```
sql = "INSERT INTO employees (id, name) VALUES (?, ?)"

with open('employees.csv') as f:
```

```

for i, line in enumerate(f):
    cursor.execute(sql, (i, line.strip()))

conn.commit()

```

For bulk insertions, `executemany()` is more efficient still:

```

data = [(1, 'Luigi'), (2, 'Mario'), (3, 'Anna')]
cursor.executemany("INSERT INTO employees (id, name) VALUES (?, ?)", data)
conn.commit()

```

3.5.2 Preventing SQL Injection

Prepared statements are essential for **security**. Building a query by concatenating strings is vulnerable to SQL injection:

```

# DANGEROUS: vulnerable to SQL injection
user_input = "'; DROP TABLE users; --"
query = f"SELECT * FROM users WHERE name = '{user_input}'"
cursor.execute(query) # Executes: SELECT * FROM users WHERE name = ''; DROP TABLE users
; --'

```

Placeholders neutralize malicious input by treating it as data rather than SQL code:

```

# SAFE: the input is escaped automatically
cursor.execute("SELECT * FROM users WHERE name = ?", (user_input,))

```

Golden rule. Never concatenate user input directly into a SQL query. Always use placeholders and pass the values as separate parameters.

3.6 Data Type Mapping

Conversion between Python types and SQL/ODBC types happens automatically, following rules the driver defines.

3.6.1 From Python to SQL

Python type	Description	ODBC/SQL type
<code>None</code>	null value	NULL
<code>str</code>	Unicode string	VARCHAR / NVARCHAR
<code>bytes</code>	binary data	VARBINARY
<code>bool</code>	boolean	BIT
<code>int</code>	arbitrary-size integer	BIGINT
<code>float</code>	64-bit floating point	DOUBLE / FLOAT
<code>decimal.Decimal</code>	arbitrary-precision decimal	DECIMAL / NUMERIC
<code>datetime.date</code>	date	DATE
<code>datetime.time</code>	time	TIME
<code>datetime.datetime</code>	timestamp	DATETIME / TIMESTAMP

3.6.2 From SQL to Python

SQL type	Variants	Python type
NULL	—	None
Character	CHAR, VARCHAR, TEXT	str
Unicode	NCHAR, NVARCHAR, NTEXT	str
Binary	BINARY, VARBINARY, BLOB	bytes
Boolean	BIT	bool
Integer	TINYINT, SMALLINT, INT, BIGINT	int
Floating	REAL, FLOAT, DOUBLE	float
Decimal	DECIMAL, NUMERIC, MONEY	decimal.Decimal
Date	DATE	datetime.date
Time	TIME	datetime.time
Timestamp	DATETIME, TIMESTAMP	datetime.datetime

3.7 Metadata Access

Beyond the data itself, querying the database's structure is often necessary: the list of tables, the column schema, the primary keys. pyodbc provides dedicated methods for this.

3.7.1 Result Set Metadata

The `cursor.description` attribute returns a tuple with information about each column of the result set:

```
cursor.execute("SELECT product_id, name, price FROM products")
for col in cursor.description:
    name, type_code, display_size, internal_size, precision, scale, nullable = col
    print(f"{name}: type={type_code}, nullable={nullable}")
```

3.7.2 Table Catalog

The `tables()` method enumerates the database's tables:

```
# All user tables
for t in cursor.tables(tableType='TABLE'):
    print(f"{t.table_schem}.{t.table_name}")

# Filter by pattern (SQL wildcard)
for t in cursor.tables(table='sales%'):
    print(t.table_name) # Tables starting with 'sales'
```

Each returned row holds: `table_cat` (catalog), `table_schem` (schema), `table_name`, and `table_type` (TABLE, VIEW, SYSTEM TABLE).

3.7.3 Column Schema

The `columns()` method returns detailed information about a table's columns:

```
for col in cursor.columns(table='products'):
    print(f"{col.column_name}: {col.type_name}({col.column_size}), "
          f"nullable={col.nullable}")
```

Available attributes include: `column_name`, `data_type` (numeric code), `type_name` (readable name), `column_size`, `decimal_digits`, `nullable`, `ordinal_position`, `column_def` (default), and others.

3.8 Project Setup: Python Environment

The following material is a practical reference for the exam project: a project layout, the dependencies, a configuration template, and reusable patterns for reading, cleaning, and enriching data with the DB-API tools introduced above.

Requirements: Python 3.8+, pip, a terminal.

Recommended IDEs: VS Code with the Python extension, PyCharm, Jupyter Notebook.

3.8.1 Project Structure and Dependencies

```
project/
|-- data/
|   |-- raw/           # Original data (tracks.json, artists.xml)
|   |-- processed/     # Cleaned data
|   |-- enriched/      # Data enriched from API
|   |-- warehouse/     # CSV ready for DW loading
|-- cache/             # API call cache
|-- src/
|   |-- cleaning.py    # Cleaning functions
|   |-- enrichment.py  # Enrichment via API
|   |-- warehouse_prep.py # CSV preparation for DW
|   |-- db_loader.py   # Loading into SQL Server
|-- config.py          # Configuration and credentials
|-- requirements.txt
```

requirements.txt:

```
requests>=2.28.0
pyodbc>=4.0.35
musicbrainzngs>=0.7.1
```

Installation:

```
python -m venv venv
source venv/bin/activate    # Linux/Mac
venv\Scripts\activate       # Windows
pip install -r requirements.txt
```

3.8.2 Configuration Template

```
# config.py
DB_CONFIG = {
    'server': 'your_server.database.windows.net',
    'database': 'Group_XX_DB',
    'username': 'your_username',
    'password': 'your_password',
    'driver': '{ODBC Driver 17 for SQL Server}'
}

API_CONFIG = {
    'musicbrainz_app': 'MyApp/1.0 (contact@email.com)',
    'rate_limit_delay': 1.0 # seconds between calls
}

PATHS = {
    'raw_tracks': 'data/raw/tracks.json',
    'raw_artists': 'data/raw/artists.xml',
    'cache_dir': 'cache/',
    'output_dir': 'data/warehouse/'
}
```

3.8.3 Data Reading Patterns

JSON:

```
import json

def load_json(filepath):
    with open(filepath, 'r', encoding='utf-8') as f:
        data = json.load(f)
        # Handles optional root key
        if isinstance(data, dict) and len(data) == 1:
            return list(data.values())[0]
        return data
```

XML:

```
import xml.etree.ElementTree as ET

def load_xml(filepath, record_tag='record'):
    tree = ET.parse(filepath)
    root = tree.getroot()
    records = []
    for elem in root.findall(f'://{record_tag}'):
        record = {child.tag: child.text for child in elem}
        records.append(record)
    return records
```

3.8.4 Data Cleaning Patterns

```
import re
from datetime import datetime

def clean_string(value, default=''):
    return str(value).strip() if value else default

def clean_numeric(value, default=None, as_type=float):
    if value is None or str(value).strip() == '':
        return default
    try:
        return as_type(value)
    except (ValueError, TypeError):
        return default

def clean_date(date_str, fmt='%Y-%m-%d'):
    """Normalize dates. Handles partial formats (year only)."""
    if not date_str:
        return None
    date_str = str(date_str).strip()
    # Try common formats
    for f in ['%Y-%m-%d', '%Y/%m/%d', '%d-%m-%Y', '%Y']:
        try:
            return datetime.strptime(date_str, f).strftime('%Y-%m-%d')
        except ValueError:
            continue
    return None
```

3.8.5 Cached API Pattern

```
import requests
import json
```

```

import os
import hashlib
import time

class CachedAPIClient:
    def __init__(self, cache_dir, rate_limit=1.0):
        self.cache_dir = cache_dir
        self.rate_limit = rate_limit
        self.last_call = 0
        os.makedirs(cache_dir, exist_ok=True)

    def _cache_path(self, key):
        h = hashlib.md5(key.encode()).hexdigest()
        return os.path.join(self.cache_dir, f"{h}.json")

    def get(self, url, params=None):
        cache_key = url + str(params)
        path = self._cache_path(cache_key)

        # Cache hit
        if os.path.exists(path):
            with open(path, 'r') as f:
                return json.load(f)

        # Rate limiting
        elapsed = time.time() - self.last_call
        if elapsed < self.rate_limit:
            time.sleep(self.rate_limit - elapsed)

        # API call
        response = requests.get(url, params=params)
        self.last_call = time.time()

        if response.status_code == 200:
            data = response.json()
            with open(path, 'w') as f:
                json.dump(data, f)
            return data
        return None

```

4 Microsoft SQL Server

Microsoft SQL Server is one of the most widely deployed data management platforms in the enterprise. It combines relational database, data warehousing, business intelligence, and advanced analytics into a single coherent ecosystem, available both on-premises and in the Azure cloud.

4.1 Editions and Licensing

SQL Server ships in several editions, each designed for a different combination of scale, feature set, and budget:

Enterprise: aimed at mission-critical workloads. It offers advanced capabilities: in-memory OLTP, compression, unlimited partitioning, Always On for high availability, and integrated analytics. It imposes no hardware resource limits.

Standard: the core data management and business intelligence features. Its limits are 128 GB of RAM for the buffer pool and 24 cores. It suits departmental workloads and mid-tier

applications.

Express: the free entry-level version. Its limits are 10 GB of storage per database, 1 GB of RAM, and 4 cores. It suits desktop applications, prototypes, and small servers.

Developer: a full replica of the Enterprise edition, free for development and testing only. It cannot be used in production.

The cloud editions (Azure SQL Database, Azure SQL Managed Instance) offer pay-as-you-go pricing with elastic scaling.

4.2 Architecture of the Suite

The SQL Server ecosystem groups specialized components that span the entire business intelligence stack:

Function	On-premises	Cloud / Self-service
Database Engine	SQL Server	Azure SQL Database
Data integration	SSIS	Azure Data Factory, Power Query
OLAP analysis	SSAS	Azure Analysis Services, Power Pivot
Reporting	SSRS	Power BI, Paginated Reports
Data Quality	DQS	—
Master Data	MDS	—

4.2.1 Core Components

- **SQL Server Database Engine:** the core relational engine that manages storage, query processing, transactions, and security.
- **SQL Server Integration Services (SSIS):** a platform for orchestrating ETL flows and moving data between heterogeneous sources (see Part 5).
- **SQL Server Analysis Services (SSAS):** an engine for multidimensional OLAP processing (cubes) and in-memory tabular models (see Part 7).
- **SQL Server Reporting Services (SSRS):** a platform for creating, managing, and distributing structured, paginated reports.
- **Master Data Services (MDS):** a solution for centrally managing an organization’s master data (customers, products, hierarchies), guaranteeing a “single version of truth”.
- **Data Quality Services (DQS):** tools for knowledge-driven data cleansing and matching.

4.2.2 Development and Administration Tools

SQL Server Management Studio (SSMS): an integrated environment for administration and development. It connects to every component of the platform (Database Engine, Analysis Services, Integration Services, Reporting Services). Its main features are the Object Explorer for navigating objects, the Query Editor with IntelliSense, the Activity Monitor for tuning, and backup/restore tools.

SQL Server Data Tools (SSDT): an extension of Visual Studio for developing database projects (schema, stored procedures, deployment), SSIS packages, SSAS models, and SSRS reports. It supports source control and CI/CD.

Azure Data Studio: a lightweight cross-platform editor optimized for queries and notebooks. It is extensible through plugins for PostgreSQL, Kubernetes, and others.

Resources

Official documentation: <https://docs.microsoft.com/sql>

Learn: <https://docs.microsoft.com/learn/paths/get-started-querying-with-transact-sql>

4.3 System Databases

SQL Server maintains four system databases that the instance needs to run. They are created automatically during installation and must never be deleted.

master: the central catalog of the instance. It holds metadata on every user database, logins and credentials, server configurations, linked servers, and endpoints. Its integrity is critical: if **master** is corrupted, the instance does not start. It requires regular backups.

model: the template for creating new databases. Any change to **model** (options, filegroups, objects) propagates automatically to databases created afterward, which helps standardize organization-wide configurations.

msdb: hosts the metadata of SQL Server Agent: scheduled jobs, alerts, operators, and execution history. It also holds SSIS packages deployed in the legacy Package Store and the Database Mail configuration.

tempdb: a temporary database recreated at every instance restart (it does not persist). It serves local temporary tables (**#temp**) and global ones (**##temp**), table variables, cursors, sort and hash operations that spill out of memory, and the version store for snapshot isolation. Its performance affects the whole server.

4.4 Namespace and Object Identification

4.4.1 Name Hierarchy

SQL Server organizes objects along a four-level hierarchy:

`[server.][database.][schema.]object`

- **Server:** the SQL Server instance (relevant for linked servers and distributed queries).
- **Database:** a logical container of objects. Every connection has a current database.
- **Schema:** a namespace inside the database that groups related objects. The default schema is **dbo** (database owner).
- **Object:** a table, view, stored procedure, function, and so on.

4.4.2 Name Resolution

When parts of a name are omitted, SQL Server applies default values:

```
-- Use current database, user's default schema
SELECT * FROM customers

-- Specify the schema explicitly
SELECT * FROM dbo.customers
```



```
-- Specify database and schema
SELECT * FROM sales_db.dbo.customers

-- Fully qualified name (for linked server)
SELECT * FROM REMOTE_SERVER.sales_db.dbo.customers
```

Best practice. Always qualifying objects with their schema (`dbo.table`) improves performance (it avoids the lookup in the user's schema) and prevents ambiguity when objects of the same name exist in different schemas.

4.5 Data Import and Export

4.5.1 Import/Export Wizard

Management Studio provides a graphical wizard to transfer data between heterogeneous sources. It is reached by right-clicking the database → **Tasks** → **Import Data** or **Export Data**.

Internally the wizard generates an SSIS package that can either run immediately or be saved for reuse. The supported sources and destinations include flat files (CSV, fixed-width), Excel, Access, OLE DB providers, ODBC connections, and other SQL Server databases.

4.5.2 Bulk Insert

For massive loads from files, the `BULK INSERT` command outperforms the wizard:

```
BULK INSERT dbo.customers
FROM 'C:\data\customers.csv'
WITH (
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    FIRSTROW = 2 -- Skip header
);
```

The command-line utility `bcp` (Bulk Copy Program) offers similar functionality with greater flexibility.

4.5.3 XML Export with `FOR XML`

SQL Server supports native XML output through the `FOR XML` clause, with several formatting modes:

RAW: each row becomes a `<row>` element with its columns as attributes.

```
SELECT employee_id, first_name, last_name
FROM employees
FOR XML RAW;
```

```
<row employee_id="1" first_name="Mario" last_name="Rossi"/>
<row employee_id="2" first_name="Anna" last_name="Verdi"/>
```

AUTO: generates elements named after the tables and handles the relationships automatically.

```
SELECT d.department_name, e.first_name
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
FOR XML AUTO;
```

```
<d department_name="Sales">
  <e first_name="Mario"/>
  <e first_name="Anna"/>
</d>
```

PATH: maximum control over the XML structure through XPath.

```
SELECT
  employee_id AS "@id",
  first_name AS "name/first",
  last_name AS "name/last"
FROM employees
FOR XML PATH('employee'), ROOT('employees');
```

```
<employees>
  <employee id="1">
    <name><first>Mario</first><last>Rossi</last></name>
  </employee>
</employees>
```

4.6 Linked Servers

A **linked server** enables distributed queries against heterogeneous data sources, so that remote databases can be treated as if they were local. The abstraction runs through OLE DB providers.

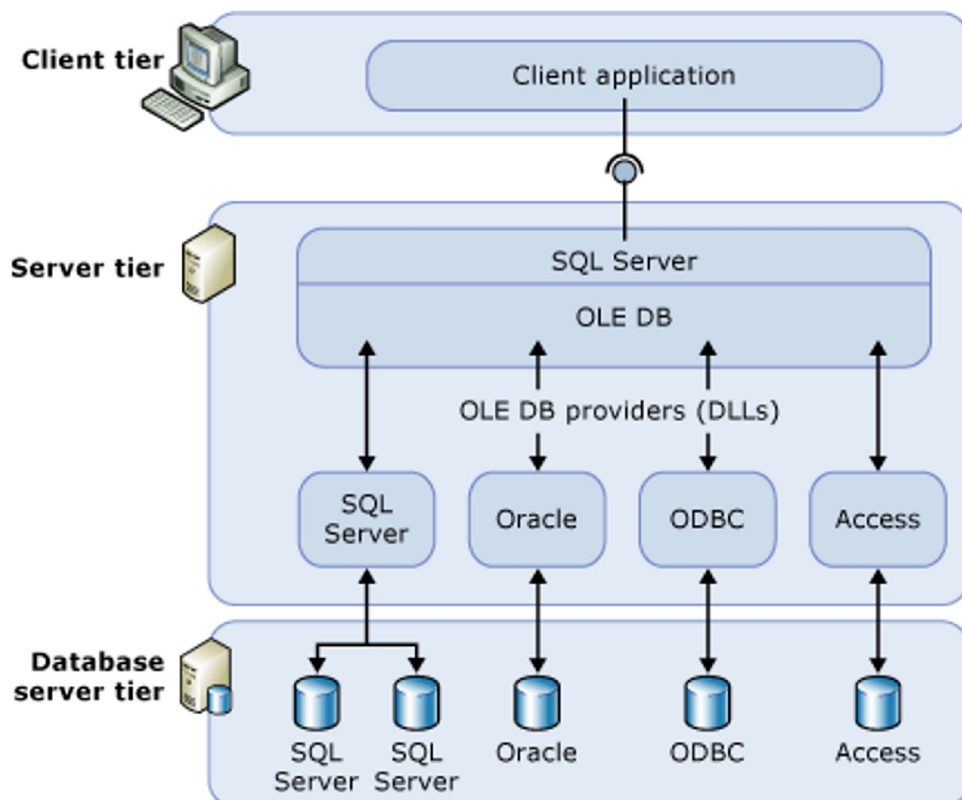


Figure 4.1. Linked server architecture: SQL Server reaches heterogeneous sources (Oracle, Access, Excel, other SQL Server instances) through OLE DB providers.

4.6.1 Configuration

A linked server is created either through SSMS (Server Objects → Linked Servers → New) or with T-SQL:

```
EXEC sp_addlinkedserver
    @server = 'ORACLE_SALES',
    @srvproduct = 'Oracle',
    @provider = 'OraOLEDB.Oracle',
    @datasrc = 'sales.example.com/ORCL';

EXEC sp_addlinkedsrvlogin
    @rmtsrvname = 'ORACLE_SALES',
    @useself = 'FALSE',
    @rmtuser = 'report_user',
    @rmtpassword = 'password';
```

4.6.2 Distributed Queries

Remote objects are referenced with the four-part notation:

`linked_server.database.schema.object`

```
-- Join between local and remote table (Oracle)
SELECT l.customer_id, l.name, r.total_orders
FROM dbo.customers AS l
JOIN ORACLE_SALES.sales.dbo.order_summary AS r
    ON l.customer_id = r.customer_id;

-- Compare data between two servers for data quality
SELECT *
FROM LOCAL_CRM.crm.dbo.contacts AS A
JOIN ORACLE_LEGACY.legacy.dbo.contacts AS B
    ON A.email = B.email
WHERE A.phone <> B.phone; -- Discrepancies in phone numbers
```

Performance. Distributed queries can be slow when the query optimizer fails to push filters down to the remote server. Check the execution plan and consider `OPENQUERY` to force remote execution:

```
SELECT * FROM OPENQUERY(ORACLE_SALES,
    'SELECT * FROM orders WHERE order_date > SYSDATE - 30');
```

4.7 Connecting to SQL Server

4.7.1 SSMS Configuration

At startup, SQL Server Management Studio's connection dialog asks for:

- **Server type:** Database Engine (for the relational engine), Analysis Services, Integration Services, or Reporting Services.
- **Server name:** the instance name in the form `hostname` (default instance) or `hostname\instance` (named instance). For Azure SQL: `server.database.windows.net`.
- **Authentication:**

- *Windows Authentication*: uses the current Windows user’s credentials (Kerberos/NTLM). It is preferable in Active Directory environments.
- *SQL Server Authentication*: a login and password stored in SQL Server. It requires the instance to be configured in mixed mode.
- *Azure Active Directory*: for Azure SQL databases with cloud identities.

4.7.2 Connection String for Applications

Applications connect through a connection string (see also Section 3.4.2):

```
-- SQL Server Authentication
Server=myserver.example.com;Database=sales;
User Id=app_user;Password=secret;

-- Windows Authentication
Server=myserver.example.com;Database=sales;
Trusted_Connection=True;

-- Azure SQL
Server=myserver.database.windows.net;Database=sales;
User Id=app_user;Password=secret;Encrypt=True;
```

5 ETL – Extract, Transform and Load

The ETL process (Extract, Transform, Load) is the bridge between operational data sources and the data warehouse: it makes sure the data arrives clean, consistent, and ready for analysis (see Section 1.2 for an introduction). This part examines the three phases of the process, the transformation operations they perform, and SQL Server Integration Services (SSIS) as an implementation platform.

5.1 Phases of the ETL Process

The ETL process runs in three sequential phases, each with its own responsibilities and its own operational difficulties.

5.1.1 Extraction

Extraction reaches into the data sources to retrieve the information to be processed. Its main difficulties come from how heterogeneous a corporate data ecosystem tends to be:

- **Source heterogeneity**: relational databases (SQL Server, Oracle, MySQL), flat files (CSV, JSON, XML), REST APIs, SOAP web services, spreadsheets, legacy systems.
- **Geographic distribution**: local sources, remote data centres, cloud services (AWS, Azure, GCP).
- **Connection protocols**: ODBC, JDBC, OLE DB, native connectors, proprietary APIs.
- **Operational constraints**: limited time windows so as not to impact the OLTP systems, network bandwidth management, authentication and authorization.

Extraction can run in **full** mode (all the data) or **incremental** mode (only the changes since the last run). Incremental mode needs a way to tell which records are new or modified: a timestamp,

a modification flag, or Change Data Capture (Section 5.4).

5.1.2 Transformation

Transformation is the heart of the ETL process. Raw data is reshaped to meet the quality, consistency, and conformance requirements of the target schema.

Transformation operations

Selection and cleaning: dropping unnecessary rows and columns, removing duplicates, handling corrupted or out-of-range values (for example a negative age, or future dates on past events).

Missing-value handling: the strategy depends on the context: substitution with a default value, statistical imputation (mean, median, mode), interpolation, or dropping the record.

Normalization: converting heterogeneous formats to a common standard: units of measure, character encodings, date formats, numeric scaling (min-max, z-score).

Splitting and merging: decomposing compound attributes (for example “Via Roma 15, Milano” into street, number, city) or combining separate fields.

Surrogate keys: generating synthetic identifiers (auto-incrementing integers) to replace the natural keys of the sources, which guarantees uniqueness inside the warehouse.

Lookup and enrichment: joining against reference tables to translate codes into descriptions, validate values, or add extra information to the records.

Aggregation: changing granularity by grouping and computing aggregate measures (for example daily transactions rolled up to monthly totals).

Derived attributes: computing new measures from existing ones (for example margin = revenue – cost, age = current date – birth date).

Record linkage: finding and reconciling records that represent the same real entity but come from different sources with different keys and formats.

Deduplication: detecting and removing duplicates through exact or fuzzy matching.

5.1.3 Loading

Loading writes the transformed data into its final destination. The mode of operation depends on the context:

- **Initial load:** the first population of the data warehouse with the entire available history. This is a one-off operation, typically run during project start-up.
- **Incremental load:** periodic update carrying only the changes since the last load. Three strategies are available:
 - *Append:* insert new records without touching the existing ones.
 - *Destructive merge:* overwrite existing records with the new values.
 - *Constructive merge:* keep history by preserving earlier versions.
- **Full refresh:** reload the whole destination. This helps when the changes are too tangled to track incrementally, or when the data needs to be fully realigned.

Staging area. The staging area is an intermediate zone where data lives temporarily while it is being processed. It isolates the ETL process from the production systems, it allows checkpoints and recovery on error, and it supports complex transformations that need more than one pass.

5.1.4 Process Orchestration

Running an ETL process means coordinating several operational concerns at once:

- **Control flow:** sets the order of task execution through precedence constraints. For example, the dimension tables must be updated before the fact table so that referential integrity holds; independent dimensions can be processed in parallel.
- **Data flow:** describes the path the data takes through the transformations. Each component takes records in, processes them, and produces output for the next component, so the components form a pipeline.
- **Error handling:** handling errors and warnings with recovery strategies: logging, notifications, automatic retries, redirecting failed records to error tables for later analysis.
- **Scheduling:** planning automatic runs: nightly batch, intraday refresh, event-driven triggers.
- **Monitoring and audit:** collecting metadata to trace data lineage, measure performance, and check the completeness and quality of each load.

5.2 SQL Server Integration Services

SQL Server Integration Services (SSIS) is Microsoft's ETL platform. It ships as part of the SQL Server suite but works on its own as well, integrating heterogeneous data sources.

5.2.1 Development Environment

SSIS projects are built in **SQL Server Data Tools (SSDT)**, which is integrated into Visual Studio. Their organization follows the usual Visual Studio hierarchy:

- **Solution:** a container of related projects, so several ETL projects can be managed as one unit.
- **Project:** a single Integration Services project holding packages, connection managers, and shared parameters.
- **Package:** an executable unit with the `.dtsx` extension, holding the full definition of one ETL process (its control flow and its data flow).

A new project is created through **File → New → Project → Integration Services Project**.

5.2.2 Control Flow

The control flow defines the package's orchestration logic through three building blocks:

Task: an atomic unit of work that performs one specific operation.

- *Execute SQL Task:* runs a query or a stored procedure.
- *Data Flow Task:* wraps a data transformation pipeline.

- *File System Task*: file operations (copy, move, delete).
- *Send Mail Task*: email notifications.
- *Script Task*: custom logic in C# or VB.NET.

Container: a logical grouping of tasks.

- *Sequence Container*: organizes related tasks.
- *For Loop Container*: iteration with a counter.
- *Foreach Loop Container*: iteration over collections (files, recordsets, variables).

Precedence Constraint: a constraint that sets the execution order between tasks. Its conditions are *Success* (proceed if the preceding task succeeds), *Failure* (proceed if it fails), and *Completion* (proceed either way). It also accepts boolean expressions for conditional branching.

5.2.3 Data Flow

The **Data Flow Task** is a special task that defines a data transformation pipeline. Inside it, three categories of component do the work:

Source: connectors that acquire data. The main types are:

- OLE DB Source, ADO.NET Source: relational databases
- Flat File Source: CSV, fixed-width files
- Excel Source: spreadsheets
- XML Source: XML documents
- Raw File Source: the binary SSIS format, used for performance

Transformation: operators that manipulate the data in transit. The most used are:

- *Derived Column*: computes new columns from expressions
- *Data Conversion*: converts data types
- *Lookup*: joins against reference tables
- *Conditional Split*: routes records conditionally
- *Merge Join*: joins two sorted flows
- *Union All*: unions several flows
- *Aggregate*: grouping and aggregate functions
- *Sort*: sorts the flow
- *Multicast*: duplicates the flow toward multiple destinations

Destination: connectors that load the data, offering the same types as the sources.

Every component exposes input and output columns that are mapped to build the flow. The **Error Output** is a special output that receives the records failing to process, so errors can be handled on their own path.

5.2.4 Type System

SSIS defines its own data-type system that acts as an intermediary between heterogeneous sources and destinations. The conversion flow is: source type → SSIS type → destination type. This keeps transformations consistent and portable across different databases.

The SSIS types cover the usual families:

- **DT_I4** (4-byte integer) and **DT_R8** (8-byte float) for numbers;
- **DT_STR**/**DT_WSTR** for ANSI and Unicode strings;
- **DT_DATE**/**DT_DBTIMESTAMP** for dates and timestamps;
- **DT_DECIMAL**/**DT_NUMERIC** for exact decimals;
- **DT_BYTES** for binary data.

5.3 Deployment and Execution

5.3.1 Debugging and Testing

While a package is being developed, SSIS offers integrated debugging tools:

- **Data Viewer**: inspects the data in transit between data-flow components, showing a sample of records so a transformation can be verified.
- **Breakpoint**: stops execution at specific points so the state can be examined.
- **Progress tab**: real-time monitoring of progress and execution statistics.

5.3.2 Deployment Modes

SSIS packages can be deployed in several ways:

- **File system**: saved as a **.dtsx** file, run locally.
- **SSIS Catalog**: deployed to a SQL Server instance in the SSISDB database, with versioning, parametrization, and centralized monitoring.
- **Package Store**: deployed to the legacy SSIS store (MSDB or a file system managed by the SSIS service).

To deploy to a server, **Project** → **Deploy** opens the deployment wizard that guides publication to the SSIS catalog.

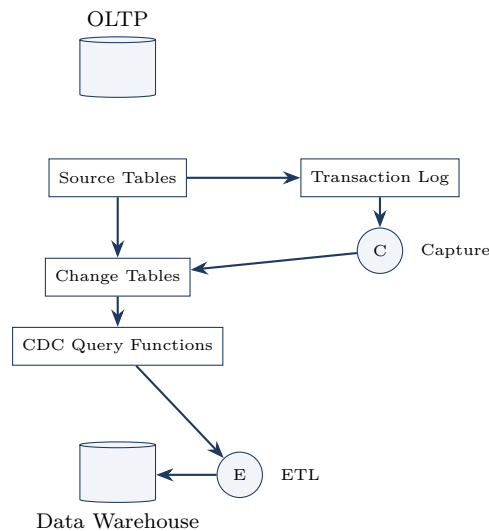
5.3.3 Execution Modes

Local execution: directly from Visual Studio (F5) during development; from the command line with the **dtexec** utility; by double-clicking the **.dtsx** file.

Server execution: from the SSISDB catalog through SQL Server Management Studio; scheduled with SQL Server Agent for automatic periodic runs; invoked from the **catalog.start_execution** stored procedure.

5.4 Change Data Capture

Change Data Capture (CDC) is a technique for identifying and capturing the changes made to source tables, which makes incremental loads far cheaper. Rather than comparing full snapshots, CDC intercepts the changes at the source.



The SQL Server CDC mechanism works as follows:

1. An asynchronous *capture process* reads the transaction log of the source database.
2. For each CDC-enabled table, it identifies the INSERT, UPDATE, and DELETE operations.
3. It populates *change tables* that record the history of changes with a timestamp and the type of operation.
4. CDC query functions (`cdc.fn_cdc_get_all_changes_*`, `cdc.fn_cdc_get_net_changes_*`) extract the changes over a time interval.
5. The ETL process queries these functions to load only the deltas, which cuts the volume of data processed dramatically.

Advantages of CDC: no impact on the source tables (no audit columns needed), full capture of every change (including intermediate ones), native support in SQL Server.

Limitations: it requires SQL Server Enterprise or Developer Edition, and it adds storage overhead for the change tables.

5.5 Slowly Changing Dimensions

Slowly Changing Dimensions (SCD) manage how the attributes of dimension tables evolve over time. Unlike transactional data, which accumulates, dimensions describe entities (customers, products, locations) whose attributes may change: a customer moves house, a product is reclassified, an employee changes department.

The strategy to adopt depends on the analytical requirement: should the data be analyzed with the current values, or with the historical values as they stood at the time of the transaction?

SCD types

Type 0: Retain Original: the attribute is never changed after the first insertion. This fits immutable values (birth date, tax code, natural IDs).

Type 1: Overwrite: the new value replaces the old one, keeping no history. Historical analyses then see the current value even for past transactions. This fits error corrections or attributes that carry no historical weight (a spelling fix, say).

Type 2: Add New Row: every change creates a new record with a distinct surrogate key. The previous record is “closed” with an end-of-validity date; the new record gets a start-of-validity date and the flag `IsCurrent = 1`. Full history is kept: past transactions stay attached to the dimension record that was valid at the time. This fits analytically relevant attributes (customer address, product category).

Type 3: Add New Column: the previous value is kept in a dedicated column (for example `PreviousAddress`). It trades space against history, and is limited to one or a few historical values.

Type 4: History Table: the current dimension is kept in one table and the full history in a separate one (a mini-dimension). This helps for dimensions with many attributes of which only a few change often.

Type 6: Hybrid: combines Type 1 + 2 + 3. Every historical record (Type 2) also carries the current value (Type 1), which makes queries that need both perspectives easier to write.

SSIS provides the **Slowly Changing Dimension Wizard**, which generates the SCD data flow automatically and can apply a different type to each attribute of the dimension.

5.6 Worked Exercises

The exercises below apply the ETL machinery of this section to concrete tasks: stratified sampling, an aggregation pipeline, a travel-sales report, weekend purchase frequency, and an SCD update.

5.6.1 Stratified Sampling

Exercise

Given a table T and a categorical column A , extract a random subset of 30% of the rows while preserving the proportion of the distinct values of A . For instance, if T contains 65% males and 35% females, the sample must keep the same proportions.

Implement the solution both in Python (with `pyodbc`) and in SSIS.

Intuition Behind the Algorithm

For each group of values of A (males, say), two counters are kept: `totRow` (rows still to examine) and `selRow` (rows still to select). The selection probability of record i is $\frac{\text{selRow}}{\text{totRow}}$, which adapts dynamically and guarantees that exactly the desired number is drawn.

Random generation. To select with probability $\frac{x}{y}$, generate a number $n \in [0, y)$ and select if $n < x$. In Python: `random.randint(0, y-1) < x` or `random.random() < x/y`.

Python Solution, Method 1: ORDER BY NEWID()

This approach delegates the randomization to the database using SQL Server's NEWID():

```
import pyodbc
import csv

# Parameters
PERCENTAGE = 0.30
table = 'census'
column = 'sex'
output_file = 'sample_newid.csv'

# Connection
conn_str = ('DRIVER={ODBC Driver 17 for SQL Server};'
            'SERVER=tcp:server.example.com;DATABASE=mydb;'
            'UID=user;PWD=password')
conn = pyodbc.connect(conn_str)
conn_inner = pyodbc.connect(conn_str)
cursor = conn.cursor()

# Distribution per stratum
cursor.execute(f"SELECT {column}, COUNT(*) FROM {table} GROUP BY {column}")

with open(output_file, 'w', newline='') as f:
    writer = csv.writer(f)
    first = True

    for col_value, n_rows in cursor:
        n_select = int(n_rows * PERCENTAGE)

        # Server-side random selection
        query = f"""SELECT TOP {n_select} *
                     FROM {table}
                     WHERE {column} = '{col_value}'
                     ORDER BY NEWID()"""

        cursor_inner = conn_inner.cursor()
        cursor_inner.execute(query)

        if first:
            writer.writerow([col[0] for col in cursor_inner.description])
            first = False

        for row in cursor_inner:
            writer.writerow(row)
        cursor_inner.close()

    cursor.close()
    conn.close()
    conn_inner.close()
```

Python Solution, Method 2: Probabilistic Algorithm

This approach makes the random selection on the client side, which helps when the server should not be burdened:

```
import pyodbc
import csv
import random

def stratified_sample(cursor, writer, total, to_select):
    """Random selection with adaptive probability."""
```

```

for row in cursor:
    prob = to_select / float(total)
    if random.random() < prob:
        writer.writerow(row)
        to_select -= 1
    total -= 1

# Parameters and connection (as above)
PERCENTAGE = 0.30
table = 'census'
column = 'sex'

conn = pyodbc.connect(conn_str)
cursor = conn.cursor()
cursor.execute(f"SELECT {column}, COUNT(*) FROM {table} GROUP BY {column}")

conn_inner = pyodbc.connect(conn_str)

with open('sample_prob.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    first = True

    for col_value, n_rows in cursor:
        n_select = int(n_rows * PERCENTAGE)

        # Retrieve ALL rows of the stratum
        cursor_inner = conn_inner.cursor()
        cursor_inner.execute(f"SELECT * FROM {table} WHERE {column} = '{col_value}'")

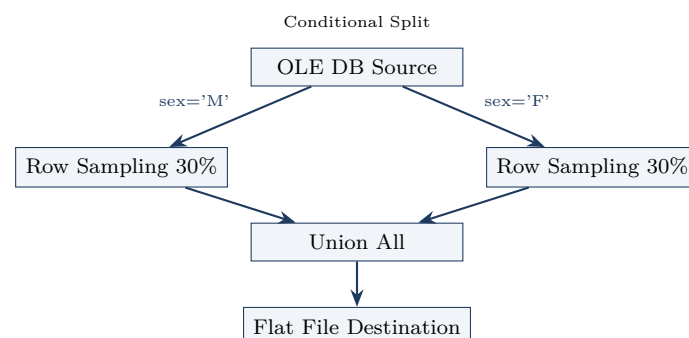
        if first:
            writer.writerow([col[0] for col in cursor_inner.description])
            first = False

        # Client-side random selection
        stratified_sample(cursor_inner, writer, n_rows, n_select)
        cursor_inner.close()

conn.close()
conn_inner.close()

```

SSIS Solution



1. OLE DB Source: `SELECT * FROM census`

2. Conditional Split:

Output: Male	Condition: <code>sex == "M"</code>
Output: Female	Condition: <code>sex == "F"</code>

3. Row Sampling (one per branch): Percentage = 30, Random seed = 42

4. **Union All:** merges the two sampled flows

5. **Flat File Destination:** census_sample_30pct.csv

Scalability. If the stratification column has many distinct values, use a Script Component that implements the probabilistic Python algorithm.

5.6.2 Aggregation Pipeline

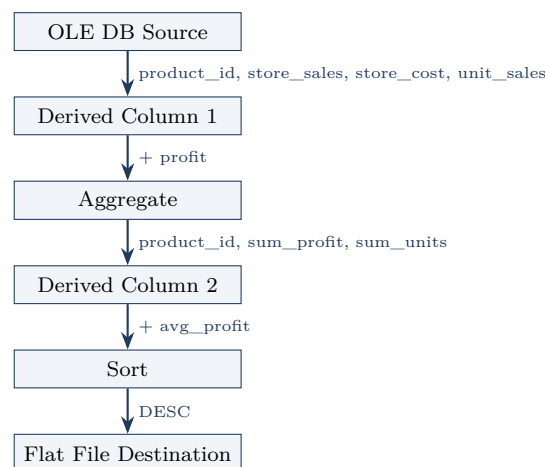
Exercise

Given the FoodMart database, design an SSIS package that produces a CSV file listing the products ordered by decreasing average profit. The profit of a single sale is `store_sales - store_cost`; the average profit is the ratio between the sum of the profits and the units sold.

Equivalent SQL Query

```
SELECT product_id
FROM sales_fact
GROUP BY product_id
ORDER BY SUM(store_sales - store_cost) / SUM(unit_sales) DESC
```

SSIS Solution



1. **OLE DB Source:**

```
SELECT product_id, store_sales, store_cost, unit_sales FROM sales_fact
```

2. **Derived Column 1:** profit of a single sale:

```
profit = store_sales - store_cost
```

3. **Aggregate:**

Input Column	Operation	Output Alias
product_id	Group by	product_id
profit	Sum	sum_profit
unit_sales	Sum	sum_units

4. Derived Column 2: average profit:

```
avg_profit = sum_units != 0 ? sum_profit / sum_units : 0
Type: DT_R8
```

5. Sort: avg_profit DESC**6. Flat File:** output_profit.csv (columns: product_id, avg_profit)*5.6.3 Travel Sales***Exercise**

A sale is classified as “travel sale” if the store of the transaction is in a different city from the customer’s residence. Produce a CSV with: the customer’s full name, total sales, and the percentage of travel sales.

*SSIS Solution***1. OLE DB Source:**

```
SELECT c.customer_id, c.fullname, c.city AS customer_city,
       st.store_city, sf.store_sales
FROM sales_fact sf
JOIN customer c ON sf.customer_id = c.customer_id
JOIN store st ON sf.store_id = st.store_id
```

2. Derived Column 1: travel flag and amount:

```
is_travel = customer_city != store_city ? 1 : 0
travel_sales = customer_city != store_city ? store_sales : 0
```

3. Aggregate:

Input	Operation	Output Alias
customer_id	Group by	customer_id
fullname	Group by	fullname
store_sales	Sum	total_sales
travel_sales	Sum	travel_sales

4. Derived Column 2: percentage:

```
pct_travel = total_sales > 0 ? (DT_R8)travel_sales / total_sales * 100 : 0
```

5. Flat File: fullname, total_sales, pct_travel*5.6.4 Weekend Purchase Frequency (FPW)***Exercise**

For each customer and month, compute the weekend purchase frequency (FPW): the number of distinct Saturday or Sunday days on which the customer made at least one purchase. Output: customer name, month/year, FPW.

Analysis

FPW = COUNT(DISTINCT weekend day with purchases). The difficulty is that the SSIS Aggregate does not support a COUNT DISTINCT under a condition. The fix is a double aggregation.

*SSIS Solution***1. OLE DB Source:**

```
SELECT sf.customer_id, t.the_date, t.the_year, t.the_month, t.day_of_week
FROM sales_fact sf
JOIN time_by_day t ON sf.time_id = t.time_id
```

2. Derived Column 1:

```
year_month = (DT_WSTR,7)(the_year) + "-" + RIGHT("0" + (DT_WSTR,2)(the_month), 2)
is_weekend = day_of_week == 1 || day_of_week == 7 ? 1 : 0
```

3. Conditional Split: Output “Weekend” if `is_weekend == 1`, discard the rest.

4. Aggregate 1: removes duplicates (same customer, same day):

Input	Operation	Output
customer_id	Group by	customer_id
year__month	Group by	year__month
the__date	Group by	the__date

5. Aggregate 2: counts the distinct days per customer/month:

Input	Operation	Output
customer_id	Group by	customer_id
year__month	Group by	year__month
the__date	Count	FPW

6. Lookup: adds fullname from the customer table

7. Flat File: fullname, year__month, FPW

*5.6.5 Q5: SCD Handling, Store 7 Becomes Store 25***Exercise**

Store 7 (type “Supermarket”) is refurbished and becomes store 25 (type “Deluxe Supermarket”). How is this Slowly Changing Dimension handled with a Type 1 and a Type 2 approach?

Type 1 Solution (Overwrite)

The record is updated and the history is lost:

```
-- Update the dimension
UPDATE store
SET store_id = 25, store_type = 'Deluxe Supermarket'
WHERE store_id = 7;
```

```
-- Update the fact table
UPDATE sales_fact SET store_id = 25 WHERE store_id = 7;
```

Consequence: every historical sale now appears as made in the Deluxe Supermarket.

Type 2 Solution (Add New Row)

The old record is closed and a new one is created, which preserves the history:

```
-- Schema with SCD Type 2
CREATE TABLE store (
    store_sk INT IDENTITY PRIMARY KEY, -- Surrogate key
    store_id INT,                      -- Business key
    store_name VARCHAR(100),
    store_type VARCHAR(50),
    valid_from DATE,
    valid_to DATE,
    is_current BIT
);

-- Close the old record
UPDATE store
SET valid_to = GETDATE(), is_current = 0
WHERE store_id = 7 AND is_current = 1;

-- Create new record
INSERT INTO store (store_id, store_name, store_type,
                  valid_from, valid_to, is_current)
SELECT 25, store_name, 'Deluxe Supermarket',
       GETDATE(), '9999-12-31', 1
FROM store WHERE store_id = 7;
```

Query that accounts for the history:

```
SELECT st.store_type, SUM(sf.store_sales) AS sales
FROM sales_fact sf
JOIN time_by_day t ON sf.time_id = t.time_id
JOIN store st ON sf.store_sk = st.store_sk
               AND t.the_date BETWEEN st.valid_from AND st.valid_to
GROUP BY st.store_type;
```

5.7 Date Dimension: A Worked Case Study

The date dimension is the most delicate one in a data warehouse. A wrong design breaks the navigability of the cube.

5.7.1 The Ambiguity Problem

Consider the hierarchy Year → Month → Day. The member “January” looks identical across every year: SSAS cannot tell January 2023 from January 2024. This produces wrong aggregations during drill-down.

5.7.2 Solution: Composite Keys

Each level needs a composite key that makes it unique:

Attribute	KeyColumn	NameColumn	Example
Year	year	year	2024
Month	year_month_int	year_month	“2024-01”
Day	ID_date	full_date	“2024-01-15”
Season	year, season	year_season	“2024-Winter”

5.7.3 Population Script

```

DECLARE @StartDate DATE = '1950-01-01';
DECLARE @EndDate DATE = '2030-12-31';

WITH DateSequence AS (
    SELECT @StartDate AS dt
    UNION ALL
    SELECT DATEADD(DAY, 1, dt) FROM DateSequence WHERE dt < @EndDate
)
INSERT INTO [date] (ID_date, full_date, [year], [month], [day],
    month_name, year_month, year_month_int,
    season, year_season, [quarter],
    day_of_week, is_weekend)
SELECT
    CONVERT(INT, FORMAT(dt, 'yyyyMMdd')),
    dt,
    YEAR(dt),
    MONTH(dt),
    DAY(dt),
    DATENAME(MONTH, dt),
    FORMAT(dt, 'yyyy-MM'),
    YEAR(dt) * 100 + MONTH(dt),
    CASE
        WHEN MONTH(dt) IN (12,1,2) THEN 'Winter'
        WHEN MONTH(dt) IN (3,4,5) THEN 'Spring'
        WHEN MONTH(dt) IN (6,7,8) THEN 'Summer'
        ELSE 'Autumn'
    END,
    CAST(YEAR(dt) AS VARCHAR(4)) + '-' +
        CASE WHEN MONTH(dt) IN (12,1,2) THEN 'Winter'
            WHEN MONTH(dt) IN (3,4,5) THEN 'Spring'
            WHEN MONTH(dt) IN (6,7,8) THEN 'Summer'
            ELSE 'Autumn' END,
    DATEPART(QUARTER, dt),
    DATEPART(WEEKDAY, dt),
    CASE WHEN DATEPART(WEEKDAY, dt) IN (1,7) THEN 1 ELSE 0 END
FROM DateSequence
OPTION (MAXRECURSION 0);

```

5.7.4 The Season Problem

Season does not sit linearly in the hierarchy: December 2024 belongs to “Winter 2025”.

Recommended solution: do not put Season inside the main hierarchy Year → Month → Day. Use Season as a separate attribute attached to Month, creating two hierarchies:

- **Calendar:** Year → Month → Day
- **Seasonal:** Year → Season (Month and Day excluded)

5.7.5 Attribute Configuration in SSAS

Attribute	KeyColumn	NameColumn	OrderBy
Year	year	year	Key
Month	year_month_int	year_month	Key
Day	ID_date	full_date	Key
Season	year + season	year_season	Key

5.8 SSIS Quick Reference

For the ETL concepts see Section 5.1. This reference gives only the syntax and the operational patterns.

5.8.1 SSIS Expression Syntax

Operators:

```
+ - * / %           -- Arithmetic
== != < > <= >=    -- Comparison
&& || !             -- Logical (AND, OR, NOT)
cond ? val1 : val2   -- Ternary
```

String functions:

```
UPPER(s)  LOWER(s)  TRIM(s)  LEN(s)
SUBSTRING(s, start, len)
REPLACE(s, old, new)
LEFT(s, n)  RIGHT(s, n)
FINDSTRING(s, search, occurrence)
```

Date functions:

```
GETDATE()
YEAR(d)  MONTH(d)  DAY(d)
DATEADD("dd", n, d)  -- Add n days
DATEDIFF("mm", d1, d2) -- Difference in months
```

Type conversions:

```
(DT_I4)expr      -- 32-bit integer
(DT_I8)expr      -- 64-bit integer
(DT_R8)expr      -- Double
(DT_WSTR, len)expr -- Unicode string
(DT_DBDATE)expr  -- Date
(DT_BOOL)expr    -- Boolean
```

NULL handling:

```
ISNULL(col)           -- TRUE if NULL
ISNULL(col) ? "default" : col -- Default pattern
REPLACENULL(col, default_value) -- Alternative
```

5.8.2 Common SSIS Patterns

Computing statistics (mean, std dev):

```
-- 1. Derived Column: square
squared = value * value
```

```
-- 2. Aggregate (GROUP BY key):
count_n = COUNT(*)
sum_val = SUM(value)
sum_sq  = SUM(squared)

-- 3. Derived Column: statistics
mean = sum_val / count_n
variance = (sum_sq / count_n) - (mean * mean)
std_dev = variance > 0 ? SQRT(variance) : 0
```

Threshold classification:

```
-- Conditional Split
LOW:      value < threshold_low
MEDIUM:  value >= threshold_low && value < threshold_high
HIGH:     value >= threshold_high
```

Manual pivot:

```
-- Derived Column before Aggregate
is_cat_A = (category == "A") ? 1 : 0
val_if_A = (category == "A") ? value : 0

-- Aggregate
count_A = SUM(is_cat_A)
total_A = SUM(val_if_A)
```

6 Data Warehousing and OLAP

A data warehouse organizes data according to a **dimensional schema** tuned for analysis, structurally different from a normalized operational database. Operational systems minimize redundancy to preserve transactional integrity, whereas dimensional modeling favors query simplicity, analytical performance, and comprehensibility for business users. The formal definition of a data warehouse and its founding properties appear in Section 1.3.

6.1 Dimensional Modeling

Dimensional modeling represents business processes through two kinds of tables: **fact tables** record the measurable events, and **dimension tables** supply the descriptive context that analysis is built on.

6.1.1 Fact Table

The **fact table** sits at the heart of the dimensional schema. Each row represents one business event (a sale, a shipment, a click) at a chosen level of granularity. Its components are the following:

- **Foreign keys:** references to the dimension tables that place the event in context (who, what, when, where).
- **Measures:** numeric values that quantify the event. They are typically additive (summable across every dimension), semi-additive (summable across some), or non-additive (ratios, percentages).

- **Surrogate key** (optional): a synthetic identifier for the fact row.

The **grain** defines exactly what a single fact row stands for. Compare “one row per product sold in each transaction” against “one row per product per store per day”. Choosing the grain is the most consequential design decision: a fine grain gives maximum analytical flexibility at the cost of storage, while an aggregated grain shrinks volumes but caps the detail that can ever be reached.

6.1.2 Dimension Tables

A **dimension table** holds the descriptive attributes that place the facts in context. Each dimension is one perspective for analysis: time, product, customer, geography, channel. These tables share a common structure:

- **Surrogate key**: an auto-incremented integer that uniquely identifies each row, independent of the natural keys of the source systems. It enables SCD handling (Section 5.5), integration of multiple sources, and fast joins.
- **Natural key**: the original code from the source system, retained for reconciliation.
- **Descriptive attributes**: textual fields used to filter, group, and label reports (product name, category, customer city).
- **Hierarchies**: parent-child relationships between attributes that define levels of aggregation (Year → Quarter → Month → Day).

6.1.3 Star Schema

The **star schema** is the foundational structure of the data warehouse. The fact table sits at the center, surrounded by dimension tables linked through foreign keys. The name comes from the star shape of the resulting ER diagram.

Its distinctive features:

- Dimension tables fully **denormalized**: every attribute of a dimension, hierarchies included, lives in a single table.
- **Simple joins**: a query needs at most one join per dimension, always between fact and dimension.
- **Intuitive queries**: the structure mirrors how users think about the data (“sales by product by month by region”).
- **Effective optimization**: DBMSs implement techniques built for star-joins (bitmap index, star transformation).

6.1.4 Snowflake Schema

The **snowflake schema** normalizes the dimensions in part, decomposing the hierarchies into separate tables. The Product dimension, for instance, splits into Product, Subcategory, and Category tables, each linked to the next.

The snowflake form is preferable when:

- Storage space is tight and dimensional redundancy is significant.

- Dimensions have very high cardinality with repeated hierarchy attributes.
- Hierarchies are shared across several dimensions (for example, a geographic hierarchy used by both Customers and Suppliers).
- Referential integrity must be enforced on the hierarchies.

The drawbacks: more complex queries with multiple joins, potentially lower performance, and reduced comprehensibility for non-technical users.

6.2 Storage Technologies

The choice of storage technology has a marked effect on analytical performance.

6.2.1 Row Storage versus Column Storage

Traditional RDBMSs store data **by row**: all attributes of a record sit contiguously on disk. This layout suits transactional operations that touch the whole record (INSERT, UPDATE, SELECT * WHERE id = x).

A **columnar database** instead stores the values of each attribute contiguously, so that every column forms its own file or segment. This layout favors analytical queries, which usually aggregate a few columns over many rows. Columnar storage brings several advantages for analytics:

- **Selective scanning**: only the attributes the query asks for are read. A query that sums sales by reading just the `amount` column over 100 million rows never loads the other 20 columns.
- **Efficient compression**: values of one type and domain compress better. Run-length encoding, dictionary encoding, and delta encoding reach ratios of 10:1 or higher on low-cardinality columns.
- **Vectorized operations**: processing contiguous vectors of values exploits the SIMD instructions of modern CPUs, handling several values per cycle.
- **Late materialization**: intermediate results can stay in columnar form, rebuilding rows only at the end.

Columnar databases include Amazon Redshift, Google BigQuery, Snowflake, ClickHouse, and Vertica. SQL Server offers the **columnstore index** to enable columnar storage on selected tables.

6.3 OLAP Concepts

OLAP (Online Analytical Processing) is the paradigm of interactive analysis over multidimensional data. The user navigates the data through intuitive operations that shift the vantage point of observation, surfacing patterns and anomalies.

6.3.1 The Data Cube

A **cuboid** is an aggregated view of the data over k dimensions. Each axis corresponds to one level of a dimensional hierarchy, and each cell holds the measure values for that combination of coordinates.

The full **data cube** comprises every cuboid obtainable from the combinations of hierarchy levels. With n dimensions, each carrying h_i levels (including the ALL level that stands for the total), the number of cuboids is

$$\prod_{i=1}^n h_i$$

For three dimensions Time (4 levels: ALL, Year, Month, Day), Product (3 levels: ALL, Category, Product), and Store (3 levels: ALL, Region, City), the count is $4 \times 3 \times 3 = 36$ distinct cuboids.

6.3.2 Navigation Operations

Drill-down: increases detail by descending the hierarchy. From sales by Year to sales by Month, for example. It expands each cell into finer subsets, raising the number of rows in the result.

Roll-up: reduces detail by climbing the hierarchy. From sales by City to sales by Region, for example. It aggregates cells into higher-level totals, cutting the number of rows.

Slice: fixes one dimension to a single value, lowering the dimensionality of the cube. Selecting only Year = 2024 turns a 3D cube into a 2D “slice”.

Dice: selects a subcube by specifying ranges or sets of values on several dimensions at once. For example: products in categories A and B, in the years 2023–2024, for stores in the North region.

Pivot (rotate): rotates the axes of the cube, rearranging the dimensions in the display. It changes only the presentation, not the data (swapping rows and columns in a pivot table, for instance).

Drill-across: navigates between different fact tables that share conformed dimensions, allowing measures from distinct processes to be compared (sales versus budget, for example).

Drill-through: reaches the transactional detail underneath an aggregated value, leaving the cube for the source data.

6.4 Cuboids in SQL

SQL Server generates cuboids through extensions of the GROUP BY clause that compute several aggregations within a single query.

6.4.1 Basic Dimensional Query

A standard dimensional query performs the **star-join** between fact table and dimension tables, groups by the desired hierarchy levels, and computes the aggregated measures:

```
SELECT L.city, P.brand, T.month,
       SUM(F.dollars_sold) AS total_sales,
       COUNT(*) AS num_transactions
FROM fact_sales AS F
  JOIN dim_location AS L ON F.location_key = L.location_key
  JOIN dim_time AS T ON F.time_key = T.time_key
  JOIN dim_product AS P ON F.product_key = P.product_key
GROUP BY L.city, P.brand, T.month;
```

This query produces a single cuboid at grain (City, Brand, Month).

6.4.2 GROUP BY CUBE

The CUBE operator generates all 2^n possible grouping combinations for n attributes, including subtotals and the grand total:

```
SELECT L.city, P.brand, T.month,
       SUM(F.dollars_sold) AS total_sales
FROM fact_sales AS F
JOIN dim_location AS L ON F.location_key = L.location_key
JOIN dim_time AS T ON F.time_key = T.time_key
JOIN dim_product AS P ON F.product_key = P.product_key
GROUP BY CUBE(L.city, P.brand, T.month);
```

With three attributes, CUBE produces $2^3 = 8$ groupings: (city, brand, month), (city, brand), (city, month), (brand, month), (city), (brand), (month), (). A NULL in a grouping column marks aggregation over that dimension.

6.4.3 GROUP BY ROLLUP

The ROLLUP operator generates the initial subsequences of the attribute list, producing $n + 1$ groupings for n attributes. It fits hierarchies with a natural ordering:

```
SELECT L.country, L.region, L.city,
       SUM(F.dollars_sold) AS total_sales
FROM fact_sales AS F
JOIN dim_location AS L ON F.location_key = L.location_key
GROUP BY ROLLUP(L.country, L.region, L.city);
```

This produces the groupings (country, region, city), (country, region), (country), (), suited to reports with progressive subtotals along a geographic hierarchy.

6.4.4 GROUPING SETS

GROUPING SETS specifies exactly which grouping combinations to compute, giving full control:

```
SELECT L.country, L.city, P.category,
       SUM(F.dollars_sold) AS total_sales
FROM fact_sales AS F
JOIN dim_location AS L ON F.location_key = L.location_key
JOIN dim_product AS P ON F.product_key = P.product_key
GROUP BY GROUPING SETS (
    (L.country, P.category), -- Sales by country and category
    (L.city),                -- Sales by city (without category)
    ()                       -- Grand total
);
```

6.4.5 The GROUPING() Function

The GROUPING() function tells genuine NULLs (missing data) apart from NULLs that signal aggregation. It returns 1 when the column is aggregated, 0 otherwise:

```
SELECT
    CASE WHEN GROUPING(L.city) = 1 THEN 'All Cities'
    ELSE L.city END AS city,
    CASE WHEN GROUPING(P.brand) = 1 THEN 'All Brands'
    ELSE P.brand END AS brand,
    SUM(F.dollars_sold) AS total_sales
FROM fact_sales AS F
JOIN dim_location AS L ON F.location_key = L.location_key
JOIN dim_product AS P ON F.product_key = P.product_key
```

```
GROUP BY CUBE(L.city, P.brand);
```

6.4.6 Implementing Slice and Dice

A **slice** is implemented with conditions in the WHERE clause:

```
SELECT L.city, P.brand, SUM(F.dollars_sold) AS total_sales
FROM fact_sales AS F
  JOIN dim_location AS L ON F.location_key = L.location_key
  JOIN dim_time AS T ON F.time_key = T.time_key
  JOIN dim_product AS P ON F.product_key = P.product_key
WHERE T.year = 2024 -- Slice: fix the time dimension
GROUP BY CUBE(L.city, P.brand);
```

A **dice** uses several conditions together:

```
WHERE T.year IN (2023, 2024)          -- Range on time
AND L.region = 'North'                -- Filter on geography
AND P.category IN ('Electronics', 'Appliances') -- Filter on product
```

6.5 OLAP Architectures

OLAP systems are classified by the underlying storage technology. The choice depends on requirements for performance, scalability, data latency, and query complexity.

6.5.1 MOLAP

Multidimensional OLAP stores data in native multidimensional structures, typically optimized sparse arrays. The cuboids are pre-computed and materialized during cube processing.

Its advantages: excellent performance on predefined queries, complex calculations pre-computed, efficient compression of the sparse arrays. Its drawbacks: processing time to build the cube, data latency (a refresh is required), limited scalability for very large cubes, and rigidity when the schema changes.

6.5.2 ROLAP

Relational OLAP keeps the data in standard relational tables, translating OLAP operations into SQL queries. Aggregations are computed on-the-fly or pre-materialized in aggregate tables.

Its advantages: high scalability (it inherits the scalability of the RDBMS), schema flexibility, real-time access to the data, no batch processing. Its drawbacks: performance that varies with the SQL optimizer, complex calculations that are less efficient, and a heavier load on the database.

6.5.3 HOLAP

Hybrid OLAP combines the two approaches: pre-computed aggregates in multidimensional form for frequent queries, detail data in relational tables for drill-through and ad-hoc analysis.

Aspect	MOLAP	ROLAP
Storage	Multidimensional arrays	Relational tables
Query performance	Excellent (pre-aggregated data)	Variable (depends on SQL)
Scalability	Limited (memory)	High
Schema flexibility	Rigid	High
Data latency	Batch refresh	Real time
Complex calculations	Efficient (pre-computed)	On-the-fly

Modern trends. In-memory systems such as SAP HANA, SQL Server with columnstore, and the tabular models of SSAS blur the traditional distinction: they keep the data in compressed columnar form in memory and compute aggregations on-the-fly with performance comparable to MOLAP, yet retain the flexibility of ROLAP.

6.6 Data Warehouse Schema: DDL and Load Order

The dimensional-modeling theory of Section 6.1 takes concrete form in the schema below: a star schema for a music-streaming warehouse, with its full DDL and the order in which its tables must be loaded. This is the schema of the course project, a warehouse built from scratch. It is distinct from the Foodmart sample database used by the worked exercises in Sections 7.8 and 5.6, which the course provides ready-made; the two illustrate the same dimensional ideas on different data.

6.6.1 Complete DDL

```
-- =====
-- DIMENSION: GEOGRAPHY
-- =====
CREATE TABLE dbo.geography (
    ID_geography    INT PRIMARY KEY,
    country         VARCHAR(50),
    country_lat     FLOAT,
    country_lon     FLOAT,
    region          VARCHAR(100),
    province        VARCHAR(60),
    city            VARCHAR(50)
);

-- =====
-- DIMENSION: DATE (SQL reserved word)
-- =====
CREATE TABLE dbo.[date] (
    ID_date         INT PRIMARY KEY,          -- YYYYMMDD format
    full_date       DATE,
    [year]          INT,
    [month]         INT,
    [day]           INT,
    month_name      VARCHAR(15),
    year_month      VARCHAR(7),              -- '2024-01' for the hierarchy
    year_month_int  INT,                     -- 202401 for ordering
    season          VARCHAR(10),
    year_season     VARCHAR(15),
    [quarter]       INT,
    day_of_week     INT,
```

```

        is_weekend      BIT
    );

-- =====
-- DIMENSION: ARTIST
-- =====
CREATE TABLE dbo.artist (
    ID_artist           VARCHAR(20) PRIMARY KEY,
    ID_birth_geography  INT NOT NULL,
    birth_date          DATE NULL,
    name                VARCHAR(50),
    type                VARCHAR(20),
    gender              VARCHAR(20),

    CONSTRAINT FK_artist_geography
        FOREIGN KEY (ID_birth_geography)
        REFERENCES dbo.geography(ID_geography)
);

-- =====
-- DIMENSION: ALBUM
-- =====
CREATE TABLE dbo.album (
    ID_album            VARCHAR(20) PRIMARY KEY,
    album_name          VARCHAR(200),
    album_type          VARCHAR(20),
    album_label         VARCHAR(50),
    ID_release_date     INT NULL,

    CONSTRAINT FK_album_date
        FOREIGN KEY (ID_release_date)
        REFERENCES dbo.[date](ID_date)
);

-- =====
-- DIMENSION: TEXT (textual features)
-- =====
CREATE TABLE dbo.[text] (
    ID_text             INT PRIMARY KEY,
    lyrics              VARCHAR(MAX),
    language            VARCHAR(10),
    explicit            BIT,
    text_complexity     FLOAT,
    n_tokens            INT,
    n_sentences         INT
);

-- =====
-- DIMENSION: SIMPHONY (audio features)
-- =====
CREATE TABLE dbo.simphony (
    ID_simphony         INT PRIMARY KEY,
    duration_s          FLOAT,
    bpm                 FLOAT,
    loudness             FLOAT,
    energy_index         FLOAT
);

-- =====
-- DIMENSION: TRACK
-- =====
CREATE TABLE dbo.track (
    ID_track            INT PRIMARY KEY,
    ID_text             INT NOT NULL,

```

```

ID_simphony      INT NOT NULL,
ID_album         VARCHAR(20) NOT NULL,
title            VARCHAR(100),
disc_number      INT,
track_number     INT,
popularity       FLOAT,

CONSTRAINT FK_track_text
    FOREIGN KEY (ID_text) REFERENCES dbo.[text](ID_text),
CONSTRAINT FK_track_simphony
    FOREIGN KEY (ID_simphony) REFERENCES dbo.simphony(ID_simphony),
CONSTRAINT FK_track_album
    FOREIGN KEY (ID_album) REFERENCES dbo.album(ID_album)
);

-- =====
-- BRIDGE TABLE: PERFORMER (M:N relationship)
-- =====
CREATE TABLE dbo.performer (
    ID_track      INT NOT NULL,
    ID_artist     VARCHAR(20) NOT NULL,
    isPrimary     BIT NOT NULL,

    CONSTRAINT PK_performer PRIMARY KEY (ID_track, ID_artist),
    CONSTRAINT FK_performer_track
        FOREIGN KEY (ID_track) REFERENCES dbo.track(ID_track),
    CONSTRAINT FK_performer_artist
        FOREIGN KEY (ID_artist) REFERENCES dbo.artist(ID_artist)
);

-- =====
-- FACT TABLE: SONG_FACT
-- =====
CREATE TABLE dbo.song_fact (
    fact_song_key INT IDENTITY(1,1) PRIMARY KEY,
    ID_track       INT NOT NULL,
    ID_upload_date INT NOT NULL,
    streams_first_month BIGINT,

    CONSTRAINT FK_fact_track
        FOREIGN KEY (ID_track) REFERENCES dbo.track(ID_track),
    CONSTRAINT FK_fact_date
        FOREIGN KEY (ID_upload_date) REFERENCES dbo.[date](ID_date)
);

```

6.6.2 Load Order

The FK constraints impose this order:

1. **No dependencies:** geography, date, text, simphony
2. **Depends on geography:** artist
3. **Depends on date:** album
4. **Depends on album, text, simphony:** track
5. **Depends on track, artist:** performer
6. **Depends on track, date:** song_fact

7 SQL Server Analysis Services

SQL Server Analysis Services (SSAS) is the OLAP engine of the Microsoft suite, built to support multidimensional analysis over large data volumes. This part covers the system architecture, the two available modelling paradigms, the development workflow, and the MDX language used to query the cubes.

7.1 SSAS Architecture

SSAS runs as a server independent of the SQL Server Database Engine. It unifies OLAP Server and Data Mining Server functionality in a single environment, and connects to heterogeneous sources through ODBC, OLE DB, flat files, and cloud services.

7.1.1 Modelling Paradigms

SSAS offers two distinct modelling paradigms, each optimised for a specific class of scenarios:

Multidimensional model: implements the classic OLAP cube structure. It requires the explicit definition of Data Source, Data Source View, Dimensions, and Cubes. Query language: MDX. Storage: MOLAP, ROLAP, or HOLAP. Best suited to complex hierarchies, sophisticated calculations, and compatibility with traditional OLAP clients.

Tabular model: relies on the in-memory VertiPaq engine with advanced columnar compression. Query and calculation language: DAX (Data Analysis Expressions). Its advantages are shorter development times, native integration with Excel and Power BI, and strong performance on medium-to-large volumes. Best suited to self-service BI and agile scenarios.

The choice between the two approaches depends on:

- **Hierarchy complexity:** the multidimensional model handles parent-child, ragged, and many-to-many hierarchies better.
- **Data volume:** the tabular model scales better thanks to VertiPaq compression.
- **Client tools:** the tabular model integrates natively with modern Power BI and Excel.
- **Team skills:** DAX has a more accessible learning curve than MDX.

7.1.2 BI Semantic Model

SSAS exposes a **unified semantic model** reachable through the XMLA (XML for Analysis) protocol. Clients connect through this standard interface regardless of the underlying technology.

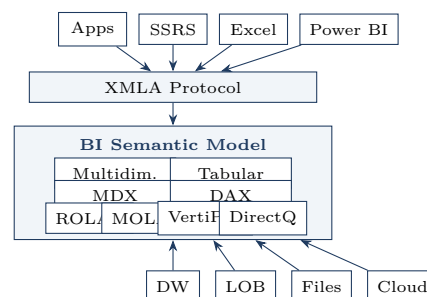


Figure 7.1. SSAS architecture: the BI Semantic Model abstracts the storage details and exposes a uniform interface to clients through XMLA.

The semantic model supports:

- Dimensions with multiple hierarchies and descriptive attributes
- Aggregate measures with calculation formulas
- Calculated metrics (calculated members) and calculated sets (named sets)
- KPIs with targets, thresholds, and visual indicators
- Interactive actions: URL, drill-through, report launch

7.2 Developing Multidimensional Projects

7.2.1 Development Environment

The development environment for SSAS is **SQL Server Data Tools (SSDT)**, integrated into Visual Studio. To create a multidimensional project: **File** → **New** → **Project** → **Analysis Services Multidimensional and Data Mining Project**.

Naming convention. In shared environments (labs, teams), prefix the project name with your own identifier in the form `<account>_<project_name>` to avoid conflicts during deployment on the same SSAS server.

7.2.2 Project Structure

An SSAS Multidimensional project is organised into components with specific responsibilities.

Data Source: defines the connections to the source databases (typically the data warehouse). Its parameters are the provider (OLE DB is recommended for compatibility), the connection string, and the impersonation credentials. For lab environments, use “Use the service account” as the impersonation option.

Data Source View (DSV): a logical view of the source tables. It allows one to:

- Rename tables and columns without altering the database
- Define logical relationships absent from the physical DB
- Create calculated columns (Named Calculations)
- Add named queries (Named Queries) as virtual tables

The DSV works in disconnected mode, storing the metadata locally.

Dimensions: definitions of the analytical dimensions. For each dimension one specifies:

- The source table from the DSV
- The key attribute (surrogate key)
- The name attribute (descriptive column shown to the user)
- The type: Standard or Time (for temporal dimensions with built-in intelligence)

Cubes: the aggregation of dimensions, measures, and relationships. A cube defines:

- Measure groups (groups of measures from the same fact table)
- Dimension usage (how dimensions connect to the measure groups)
- Calculated members and named sets
- KPIs, actions, perspectives

7.2.3 Dimension Attribute Properties

Dimension attributes expose configurable properties in the Properties panel:

- **AttributeHierarchyVisible:** when True, the attribute appears as a navigable flat hierarchy (default: True for every attribute).
- **OrderBy:** the sort criterion for members (Key, Name, AttributeKey, AttributeName).
- **DiscretizationMethod:** for continuous numeric attributes, enables automatic discretisation into buckets (None, Automatic, EqualAreas, Clusters).
- **Type:** the semantic type of the attribute. Keep “Regular” unless a specific need arises (for example “Country” to enable geographic features).
- **Usage:** the role of the attribute: Key (primary key), Regular (standard attribute), Parent (for parent-child hierarchies).

User hierarchies are defined in the **Dimension Structure** panel by dragging attributes into the Hierarchies area to build ordered levels (for example Year → Quarter → Month).

7.2.4 The Build-Deploy-Process Cycle

The SSAS development workflow has three distinct phases.

Build: a syntactic check of the project run locally. It validates the XML definitions without contacting the server. Typical errors are missing references and inconsistent relationships.

Deploy: transfers the compiled project to the target SSAS server. It is configured under **Project Properties** → **Deployment**:

- **Server:** the URL of the SSAS server (for example `localhost` or `server.domain.com`)
- **Database:** the name of the target database
- **Processing Option:** Default (process if needed), Full, DoNotProcess

After the deploy, structural changes (new dimensions, attributes, hierarchies) become visible, but the data may not yet be refreshed.

Process: populates the cube by reading the data from the sources. It computes aggregations, builds indexes, and refreshes the cache. The available modes are:

- *Process Full:* fully rebuilds the object
- *Process Update:* refreshes with the changed data (dimensions only)
- *Process Index:* rebuilds the indexes only
- *Process Data:* loads the data without computing aggregations

7.3 OLAP Storage in SSAS

SSAS supports the three standard OLAP architectures (see Section 6.5 for the general concepts), with specific configuration options.

7.3.1 Storage Modes

MOLAP: the data is copied from the data warehouse into proprietary multidimensional structures optimised for OLAP queries. Performance is excellent, but there is an update latency because

processing is required.

ROLAP: SSAS generates SQL queries against the data warehouse on every request. The data is always current, but performance depends on the underlying RDBMS.

HOLAP: pre-computed aggregates sit in MOLAP, detail data stays in ROLAP. It balances performance against data freshness.

Configuration happens at the partition level (a partition of a measure group) through the **Storage Mode** property. The available options are:

- **MOLAP:** manual or scheduled processing
- **Scheduled MOLAP:** automatic rebuild at defined intervals
- **Automatic MOLAP:** refresh triggered by changes to the source data
- **Low-latency MOLAP:** near real-time update within a configurable window
- **Real-time HOLAP:** MOLAP cache with ROLAP fallback for recent data
- **Real-time ROLAP:** direct access to the data warehouse, no cache

7.3.2 Proactive Caching

Proactive caching automates the management of the MOLAP cache in response to changes in the source data. The mechanism proceeds as follows:

1. The RDBMS notifies changes through SQL Server Notification Services or polling
2. SSAS invalidates the affected cache
3. After a **silence interval** (a waiting period that lets changes accumulate), the rebuild begins
4. During the rebuild, queries are served from the existing (stale) cache or in ROLAP mode
5. The **latency** sets the maximum tolerated time between a change and its availability in the cube

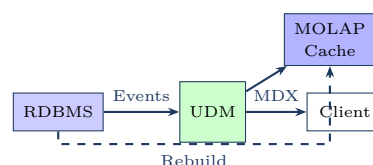


Figure 7.2. Proactive caching: the RDBMS notifies the changes, and SSAS invalidates and rebuilds the MOLAP cache automatically.

7.4 Advanced Features

7.4.1 KPIs (Key Performance Indicators)

A **KPI** combines a metric with a target value and evaluation thresholds. Its components are:

- **Value:** the measure or MDX expression to monitor
- **Goal:** the target value (constant or calculated)
- **Status:** an expression that determines the state (for example -1, 0, 1 for red/yellow/green)
- **Trend:** an expression that signals the direction (improvement or decline)

Clients render KPIs with graphical indicators (traffic lights, gauges, arrows) that show at a glance whether a metric is on track against its objectives.

7.4.2 Perspectives

A **perspective** defines a subset of the cube aimed at a specific role or use case. It selects which dimensions, hierarchies, attributes, measures, and KPIs are visible.

For example, a *Product Manager* sees only the Product and Time dimensions with sales and margin measures; a *Store Manager* accesses the Store and Time dimensions with operational measures.

Perspectives provide no security, since the excluded data remains reachable through direct MDX, but they simplify navigation by hiding the elements that do not matter.

7.4.3 Security

Roles manage the access permissions to the cube:

- **Database permissions:** Read, Process, Full Control
- **Cube permissions:** read, drill-through, write (writeback)
- **Dimension security:** a filter on the visible members of a dimension
- **Cell security:** restrictions at the single-cell level (computationally expensive)

Roles are assigned to Windows/Active Directory users or groups.

7.5 MDX: MultiDimensional eXpressions

MDX (MultiDimensional eXpressions) is the standard language for querying OLAP cubes and defining calculations. Originally developed by Microsoft, it is now adopted by most OLAP servers.

7.5.1 Query Structure

MDX syntax follows this pattern:

```
[WITH <formula_specification> [, <formula_specification> ...]]
SELECT [<axis_specification> [, <axis_specification> ...]]
FROM <cube_specification>
[WHERE < slicer_specification>]
```

Basic example:

```
SELECT [Measures].[Store Sales] ON COLUMNS,
       [Customer].[Geography].[Country].Members ON ROWS
FROM [Sales]
WHERE [Time].[Calendar].[Year].&[2024]
```

The **axes** (ON COLUMNS, ON ROWS, ON PAGES, ...) define the dimensions of the result; the **WHERE** clause acts as a slicer, fixing the context for the measures.

7.5.2 Member References

Members are identified through two syntaxes.

By key: [Dimension].[Hierarchy].[Level].&[MemberKey]

```
[Time].[Calendar].[Month].&[2024-01-01T00:00:00]
[Product].[Category].&[Bikes]
```

By name: [Dimension].[Hierarchy].[Level].[MemberName]

```
[Time].[Calendar].[Month].[January 2024]
[Geography].[Country].[Italy]
```

7.5.3 Tuples, Sets, and Axes

A **tuple** identifies a cell by giving one member for each relevant dimension:

```
([Time].[2024], [Product].[Bikes], [Measures].[Sales])
```

A **set** is an ordered collection of tuples:

```
{[Product].[Bikes], [Product].[Clothing], [Product].[Accessories]}
```

The set operators include + (union), - (difference), and * (crossjoin).

An **axis** specifies what to display along one axis of the result:

```
NON EMPTY [Product].[Category].Members ON COLUMNS
```

NON EMPTY drops the tuples whose values are all NULL.

7.5.4 Navigation Functions

MDX provides functions to navigate hierarchies:

```
[Time].[2024].Parent           -- Parent member (All)
[Time].[2024].Children         -- Children (Q1, Q2, Q3, Q4)
[Time].[2024].[Q1].Siblings    -- Siblings (Q1, Q2, Q3, Q4)
[Time].[2024].FirstChild      -- First child (Q1)
[Time].[2024].LastChild       -- Last child (Q4)

Ancestors([Time].[January 2024], 2) -- Ancestors 2 levels up
Descendants([Time].[2024], [Month])  -- Descendants at the Month level
```

7.5.5 Calculated Members and Sets

The WITH clause introduces formulas evaluated at query time.

Calculated Member:

```
WITH MEMBER [Measures].[Profit] AS
    [Measures].[Sales] - [Measures].[Cost]

WITH MEMBER [Measures].[Profit Margin] AS
    [Measures].[Profit] / [Measures].[Sales],
    FORMAT_STRING = "Percent"
```

Named Set:

```
WITH SET [Top5Products] AS
    TopCount(
        [Product].[Product Name].Members,
        5,
        [Measures].[Sales]
    )
```

7.5.6 Common MDX Functions

Aggregation:

```
Sum([Product].[Category].Members, [Measures].[Sales])
Avg([Time].[Month].Members, [Measures].[Sales])
Count([Customer].[Customer].Members)
```

Time Intelligence:

```
-- Previous period
([Time].CurrentMember.PrevMember, [Measures].[Sales])

-- Same period previous year
ParallelPeriod([Time].[Year], 1, [Time].CurrentMember)

-- Year-to-date
Sum(PeriodsToDate([Time].[Year], [Time].CurrentMember),
    [Measures].[Sales])

-- 3-month moving average
Avg({[Time].CurrentMember.Lag(2):[Time].CurrentMember},
    [Measures].[Sales])
```

Ranking and filtering:

```
-- Top 10 products by sales
TopCount([Product].[SKU].Members, 10, [Measures].[Sales])

-- Products with sales > 10000
Filter([Product].[SKU].Members,
    [Measures].[Sales] > 10000)

-- Rank of a product
Rank([Product].CurrentMember,
    Order([Product].[SKU].Members, [Measures].[Sales], BDESC))
```

7.6 SSAS Build Checklist

For the SSAS concepts see Part 7. This section gives the operational checklist.

7.6.1 Project Creation

1. **New project:** File → New → Project → Analysis Services Multidimensional
2. **Data Source:** Right-click → New Data Source → OLE DB → Credentials → Test Connection
3. **Data Source View:** New DSV → Select tables → Verify relationships → Add Named Calculations if needed
4. **Dimensions:** For each dimension → New Dimension → Wizard → Select table, key, name attribute
5. **Hierarchies:** Open the dimension → Drag attributes into the Hierarchies area
6. **Cube:** New Cube → Select fact table → Select measures → Select dimensions
7. **Deploy:** Project Properties → Deployment → Configure Server and Database
8. **Process:** Right-click the cube → Process → Full

7.6.2 Useful Named Calculations

To be added in the DSV on the date table:

```
-- Season
CASE
    WHEN month IN (6,7,8) THEN 'Summer'
    WHEN month IN (9,10,11) THEN 'Autumn'
    WHEN month IN (12,1,2) THEN 'Winter'
    ELSE 'Spring'
END

-- YearMonth (for NameColumn)
CAST(year AS VARCHAR(4)) + '-' + RIGHT('0' + CAST(month AS VARCHAR(2)), 2)
```

7.7 Power BI

Power BI is Microsoft's front-end for interactive reports and dashboards. It does not replace SSAS: it sits on top of the cube built here, querying it live so that the measures and hierarchies defined in the model drive the visuals. The rest of this section covers how to connect Power BI to the cube, which visual components map to which analytical needs, and how the visuals interact.

7.7.1 Connecting to the SSAS Cube

1. Get Data → Database → SQL Server Analysis Services
2. Server: the SSAS server address
3. **Connect live**: select this option
4. Select the cube from the list

Connect Live limitations: DAX measures cannot be created in Power BI; every measure must already exist in the SSAS cube.

7.7.2 Visual Components

Component	Configuration
Slicer	Field: dimension attribute. Vertical orientation for long lists.
Bar Chart	Axis: dimension. Values: measure. Legend: segmentation.
Line Chart	X-Axis: time. Y-Axis: measure.
Map	Location: Geography. Size: measure.
Card	A single aggregated measure.

7.7.3 Interactivity

Select a visualisation → Format → Edit Interactions → for every other visualisation choose: **Filter** (filters the data), **Highlight** (highlights it), **None**.

7.8 Worked Exercises

The following exercises work the same analytical questions two ways: once in SQL over the star schema, once in MDX over the cube. Comparing the two solutions shows where MDX earns its keep, on ranking, period-over-period comparison, and cumulative totals, and where plain SQL is already enough. Each exercise states the problem, then gives both solutions in full.

7.8.1 Q0: Sales Distribution by Quarter and City

Exercise

Compute the sales distribution by quarter and customer city, reporting: the absolute value, the percentage of the grand total, and the percentage of the customer's country.

SQL Solution

```
SELECT
    t.quarter AS quarter,
    c.city AS city,
    SUM(sf.store_sales) AS sales,
    -- Percentage of the grand total
    SUM(sf.store_sales) * 100.0 /
        SUM(SUM(sf.store_sales)) OVER () AS pct_of_total,
    -- Percentage of the country
    SUM(sf.store_sales) * 100.0 /
        SUM(SUM(sf.store_sales)) OVER (PARTITION BY c.country) AS pct_of_country
FROM sales_fact sf
JOIN time_by_day t ON sf.time_id = t.time_id
JOIN customer c ON sf.customer_id = c.customer_id
GROUP BY t.quarter, c.city, c.country
ORDER BY t.quarter, sales DESC;
```

MDX Solution

```
WITH
MEMBER [Measures].[Pct Total] AS
    [Measures].[Store Sales] /
    ([Measures].[Store Sales], [Time].[Quarter].[All]],
    [Customers].[City].[All]),
    FORMAT_STRING = "0.00%"

MEMBER [Measures].[Pct Country] AS
    [Measures].[Store Sales] /
    ([Measures].[Store Sales],
    ANCESTOR([Customers].CurrentMember, [Customers].[Country])),
    FORMAT_STRING = "0.00%"

SELECT
    {[Measures].[Store Sales],
    [Measures].[Pct Total],
    [Measures].[Pct Country]} ON COLUMNS,
    NON EMPTY
        [Time].[Quarter].Members * [Customers].[City].Members ON ROWS
FROM [Sales]
```

7.8.2 Q1: Top 5 Product Categories

Exercise

Find the top 5 product categories by sales. Report: sales, units sold, distinct customers.
Filters: Q1 1998, state of California.

SQL Solution

```
SELECT TOP 5
    p.product_category,
    SUM(sf.store_sales) AS sales,
    SUM(sf.unit_sales) AS units,
    COUNT(DISTINCT sf.customer_id) AS distinct_customers
FROM sales_fact sf
JOIN time_by_day t ON sf.time_id = t.time_id
JOIN customer c ON sf.customer_id = c.customer_id
JOIN product p ON sf.product_id = p.product_id
WHERE t.quarter = 'Q1' AND t.the_year = 1998
    AND c.state_province = 'CA'
GROUP BY p.product_category
ORDER BY sales DESC;
```

MDX Solution

```
WITH
MEMBER [Measures].[Distinct Customers] AS
    DISTINCTCOUNT(EXISTING [Customers].[Customer ID].Members)

SELECT
    {[Measures].[Store Sales],
     [Measures].[Unit Sales],
     [Measures].[Distinct Customers]} ON COLUMNS,
    TOPCOUNT(
        [Product].[Product Category].Members,
        5,
        [Measures].[Store Sales]
    ) ON ROWS
FROM [Sales]
WHERE (
    [Time].[Quarter].[Q1 1998],
    [Customers].[State Province].[CA]
)
```

7.8.3 Q2: Most Profitable Stores

Exercise

Analyse the stores by average profit per customer and per basket. Include the month-over-month (MoM) variation.

SQL Solution

```
WITH StoreProfits AS (
    SELECT
        st.store_id, st.store_name,
        t.quarter, t.the_month,
        SUM(sf.store_sales - sf.store_cost) AS profit,
        COUNT(DISTINCT sf.customer_id) AS n_customers,
        COUNT(DISTINCT CONCAT(sf.customer_id, '-', sf.time_id)) AS n_baskets
```

```

FROM sales_fact sf
JOIN store st ON sf.store_id = st.store_id
JOIN time_by_day t ON sf.time_id = t.time_id
GROUP BY st.store_id, st.store_name, t.quarter, t.the_month
),
WithAvgs AS (
    SELECT *,
        profit / NULLIF(n_customers, 0) AS profit_per_customer,
        profit / NULLIF(n_baskets, 0) AS profit_per_basket,
        LAG(profit) OVER (PARTITION BY store_id ORDER BY the_month) AS prev_profit
    FROM StoreProfits
)
SELECT store_name, quarter, the_month,
    profit_per_customer, profit_per_basket,
    CASE WHEN prev_profit > 0
        THEN (profit - prev_profit) * 100.0 / prev_profit
        ELSE NULL END AS mom_variation_pct
FROM WithAvgs
ORDER BY profit_per_customer DESC;

```

MDX Solution

```

WITH
MEMBER [Measures].[Profit] AS
    [Measures].[Store Sales] - [Measures].[Store Cost]

MEMBER [Measures].[Customer Count] AS
    DISTINCTCOUNT([Customers].[Customer ID].Members)

MEMBER [Measures].[Profit Per Customer] AS
    IIF([Measures].[Customer Count] = 0, NULL,
        [Measures].[Profit] / [Measures].[Customer Count])

MEMBER [Measures].[Prev Month Profit] AS
    ([Measures].[Profit], [Time].[Month].PrevMember)

MEMBER [Measures].[MoM Variation] AS
    IIF(ISEMPTY([Measures].[Prev Month Profit]) OR
        [Measures].[Prev Month Profit] = 0,
        NULL,
        ([Measures].[Profit] - [Measures].[Prev Month Profit]) /
        [Measures].[Prev Month Profit]
    ),
    FORMAT_STRING = "0.00%"

SELECT
    {[Measures].[Profit Per Customer], [Measures].[MoM Variation]} ON COLUMNS,
    NON EMPTY
        [Store].[Store Name].Members *
        [Time].[Quarter].Members *
        [Time].[Month].Members ON ROWS
FROM [Sales]
ORDER BY [Measures].[Profit Per Customer] DESC

```

7.8.4 Q3: Top 5 Categories by Margin per Age Bracket

Exercise

For each customer age bracket, find the 5 product categories with the best percentage margin (profit/sales).

SQL Solution

```
WITH CategoryMargins AS (
    SELECT
        c.age_bracket,
        p.product_category,
        SUM(sf.store_sales - sf.store_cost) * 100.0 /
            NULLIF(SUM(sf.store_sales), 0) AS margin_pct,
        ROW_NUMBER() OVER (
            PARTITION BY c.age_bracket
            ORDER BY SUM(sf.store_sales - sf.store_cost) /
                NULLIF(SUM(sf.store_sales), 0) DESC
        ) AS rn
    FROM sales_fact sf
    JOIN customer c ON sf.customer_id = c.customer_id
    JOIN product p ON sf.product_id = p.product_id
    GROUP BY c.age_bracket, p.product_category
)
SELECT age_bracket, product_category, margin_pct
FROM CategoryMargins
WHERE rn <= 5
ORDER BY age_bracket, margin_pct DESC;
```

MDX Solution

```
WITH
MEMBER [Measures].[Margin Pct] AS
    IIF([Measures].[Store Sales] = 0, NULL,
        ([Measures].[Store Sales] - [Measures].[Store Cost]) /
        [Measures].[Store Sales]
    ),
FORMAT_STRING = "0.00%"

SELECT
    [Measures].[Margin Pct] ON COLUMNS,
    GENERATE(
        [Customers].[Age Bracket].Members,
        TOPCOUNT(
            [Product].[Product Category].Members,
            5,
            ([Measures].[Margin Pct], [Customers].[Age Bracket].CurrentMember)
        ) * {[Customers].[Age Bracket].CurrentMember}
    ) ON ROWS
FROM [Sales]
```

7.8.5 Q4: Quantity Sold with Monthly Variation

Exercise

Compute the quantity (Kg) sold for each month of 1998, with the percentage variation against the previous month.

SQL Solution

```

WITH MonthlyKg AS (
    SELECT
        t.quarter, t.the_month,
        SUM(sf.unit_sales * p.gross_weight) AS kg_sold
    FROM sales_fact sf
    JOIN time_by_day t ON sf.time_id = t.time_id
    JOIN product p ON sf.product_id = p.product_id
    WHERE t.the_year = 1998
    GROUP BY t.quarter, t.the_month
)
SELECT
    quarter, the_month, kg_sold,
    LAG(kg_sold) OVER (ORDER BY the_month) AS prev_month_kg,
    CASE WHEN LAG(kg_sold) OVER (ORDER BY the_month) > 0
        THEN (kg_sold - LAG(kg_sold) OVER (ORDER BY the_month))
            * 100.0 / LAG(kg_sold) OVER (ORDER BY the_month)
        ELSE NULL END AS mom_variation_pct
FROM MonthlyKg
ORDER BY the_month;

```

MDX Solution

```

WITH
MEMBER [Measures].[Kg Sold] AS
    [Measures].[Unit Sales] * [Product].[Gross Weight]

MEMBER [Measures].[Prev Month Kg] AS
    ([Measures].[Kg Sold], [Time].[Month].PrevMember)

MEMBER [Measures].[MoM Var Pct] AS
    IIF(ISEMPTY([Measures].[Prev Month Kg]) OR
        [Measures].[Prev Month Kg] = 0,
        NULL,
        ([Measures].[Kg Sold] - [Measures].[Prev Month Kg]) /
        [Measures].[Prev Month Kg]
    ),
    FORMAT_STRING = "0.00%"

SELECT
    {[Measures].[Kg Sold],
    [Measures].[Prev Month Kg],
    [Measures].[MoM Var Pct]} ON COLUMNS,
    [Time].[Month].Members ON ROWS
FROM [Sales]
WHERE [Time].[Year].[1998]

```

*7.8.6 Q1extra: Top 5 Categories by YTD Sales***Exercise**

Find the top 5 categories by cumulative (Year-To-Date) sales from 1 January 1998 in California, evaluated at March 1998.

MDX Solution

```

WITH
MEMBER [Measures].[YTD Sales] AS
    SUM(
        PERIODSTODATE([Time].[Year], [Time].[Month].[March 1998]),

```



```

        [Measures].[Store Sales]
    )

SELECT
    [Measures].[YTD Sales] ON COLUMNS,
    TOPCOUNT(
        [Product].[Product Category].Members,
        5,
        [Measures].[YTD Sales]
    ) ON ROWS
FROM [Sales]
WHERE [Customers].[State Province].[CA]

```

7.8.7 Qextra: Cities per Region with Sales > \$4000

Exercise

How many cities per region exceeded \$4,000 in sales in March 1998?

SQL Solution

```

SELECT region, COUNT(*) AS n_cities_over_4000
FROM (
    SELECT c.state_province AS region, c.city,
           SUM(sf.store_sales) AS city_sales
    FROM sales_fact sf
    JOIN time_by_day t ON sf.time_id = t.time_id
    JOIN customer c ON sf.customer_id = c.customer_id
    WHERE t.the_month = 3 AND t.the_year = 1998
    GROUP BY c.state_province, c.city
    HAVING SUM(sf.store_sales) > 4000
) AS CityAbove4000
GROUP BY region;

```

MDX Solution

```

WITH
MEMBER [Measures].[Cities Over 4000] AS
    COUNT(
        FILTER(
            EXISTING [Customers].[City].Members,
            [Measures].[Store Sales] > 4000
        )
    )

SELECT
    [Measures].[Cities Over 4000] ON COLUMNS,
    [Customers].[State Province].Members ON ROWS
FROM [Sales]
WHERE [Time].[Month].[March 1998]

```

7.9 MDX Pattern Reference

A quick-reference table for the most common MDX patterns:

Need	MDX Pattern
% of total	Measure / (Measure, Dim.[(All)])
% of parent	Measure / (Measure, ANCESTOR(CurrentMember, Level))
Top N	TOPCOUNT(Set, N, Measure)
Bottom N	BOTTOMCOUNT(Set, N, Measure)
Count distinct	DISTINCTCOUNT(Set)
Previous month	(Measure, [Time].[Month].PrevMember)
Previous year	(Measure, PARALLELPERIOD([Time].[Year], 1))
Cumulative YTD	SUM(PERIODSTODATE([Time].[Year]), Measure)
% variation	(Curr - Prev) / Prev with IIF to handle NULL/zero
Filter on a measure	FILTER(Set, Condition)
Top N per group	GENERATE(OuterSet, TOPCOUNT(InnerSet, N, Measure))
Moving average	AVG({[Time].CurrentMember.Lag(N): [Time].CurrentMember}, Measure)
Non-empty members	NON EMPTY Set ON AXIS or NONEMPTY(Set, Measure)
Existing context	EXISTING Set: filters by the current context